sendo I a matriz identidade $(p \times p)$.

A equação (3.2) pode ser desenvolvida do seguinte modo:

$$\begin{array}{rcl} \det(J) & = & det(J^0)det(I+\epsilon(J^0))^{-1}\Delta J \\ & = & det(J^0) + \epsilon Tr\left(det(J^0)(J^0)^{-1}\Delta J\right) + \phi(\epsilon) \end{array} \tag{3.3}$$

Como $det(J^0)(J^0)^{-1} = adj(J^0)$, então, a equação (3.6), utilizando-se as equações (3.1-3.3), pode ser expressa por:

$$det(J) = det(J^0) + \epsilon \left\langle \psi(adj(J^0)^T), \psi(\Delta J) \right\rangle + \phi(\epsilon)$$

sendo adj(J^0) a matriz adjunta de J^0 , *i.e.*, adj(J^0) = cof$((J^0)^T)$.

A partir da equação (3.2), conclui-se que a sensibilidade da função $det(J^0)$ pode ser expressa por :

$$\partial det(J)/\partial J = \psi(adj(J)^T)$$

Dividindo-se a equação (3.2) por $det(J^0)$, obtém-se:

$$\tau = 1 + \Delta\tau + \phi(\epsilon) \tag{3.4}$$

sendo : τ a função determinante normalizada, ou seja $\tau \ \underline{\Delta} \frac{\det(J)}{\det(J^0)}$, $\psi(\epsilon) = \frac{\phi(\epsilon)}{\det(J^0)}$, e $\Delta\tau = \epsilon \left\langle \psi(((J^0))^{-1})^T, \psi(\Delta J) \right\rangle$.

Também, a partir da equação (3.4), conclui-se que:

$$\left(\partial det\left(J^0\right)/\partial J|_{J=J^0}\right) = \psi(((J^0)^{-1})^T) \tag{3.5}$$

A equação (3.5) representa a sensibilidade da função determinante normalizada. Assim, fazendo-se $a = \psi\{((J^0)^{-1})^T\}$ (vetor) e $C\underline{\Delta}\psi[(J^0)^{-1}]^T = \begin{bmatrix} C_H & C_N \\ C_M & C_L \end{bmatrix}$ obtém-se a seguinte representação para a que é a sensibilidade da função determinante normalizada, na forma vetorial:

$$a = [C_{11} \cdots C_{p1} \cdots C_{1p} \cdots C_{pp}]^T .$$

A variação linear da função determinante normalizada $\Delta\tau$ pode ser expressa por:

$$\Delta\tau = \epsilon \sum_{i=1}^{p} \sum_{j=1}^{p} C_{ij}\Delta J_{ij}$$

Deve-se observar que a matriz ΔJ é uma estrutura esparsa, semelhante à esparsidade da matriz Ybarra. Assim, poucos são os elementos de ΔJ que são diferentes de zero.

Desconsiderando-se os termos não-lineares $\phi(\epsilon)$, a partir da equação (3.4), fazendo-se $\tau = 0$ e $\epsilon = 1$, obtém-se:

$$\langle a, \psi(\Delta J) \rangle = -1. \tag{3.6}$$

A equação (3.6) estabelece a relação entre os vetores $\psi\{((J^0)^{-1})^T\}$ e $\psi(\Delta)$ tais que reproduzem, considerando-se o modelo linear, um valor nulo para a função $\det(J)$, ou seja, uma condição limite para a estabilidade de tensão. Deste modo, a partir desta equação, pode-se estimar a distância euclidiana que corresponde à margem de segurança ou a distância em que o ponto de operação do sistema encontra-se em relação à condição limítrofe da estabilidade estática:

$$\mathcal{M} = \sqrt{\frac{1}{\langle a, a\rangle}} \tag{3.7}$$

correspondente a uma solução da equação (3.6) para $\psi(\Delta J) = -\dfrac{a}{\langle a, a\rangle}$.

A margem de segurança $\mathcal{M}$, calculada através da equação (3.7), corresponde a uma solução de mínima norma.

A escolha da representação dada pela equação (3.4), tem sido importante, isto porque permite realizar as análises levando-se em conta a estrutura real do sistema (sem redução).

Definição 3.1. *Autovalor crítico* ($\lambda_{min}(J^0)$) *é definido como sendo o menor autovalor positivo da matriz jacobiana* J^0.

O autovalor crítico (definição 3.1) corresponde ao autovalor de interesse para análise da estabilidade de tensão, tendo em vista que é aquele que indica a máxima aproximação do colapso de tensão e, portanto, apresenta maior possibilidade de se tornar nulo à medida que houver o aumento da demanda de energia elétrica.

A partir da álgebra matricial, pode-se chegar ao seguinte resultado [5, 2]:

$$\lambda_{min}(J^0) \geq \mathcal{M} \geq 0.$$

Deste modo, a margem de segurança $\mathcal{M}$ representa uma estimativa pessimista (favorável à segurança do sistema) do parâmetro $\lambda_{min}(J^0)$. A estimativa de $\lambda_{min}(J^0)$ pode ser calculada, de forma alternativa, aplicando-se a norma matricial inversa de J^0. Este resultado é similar ao apresentado na referência [8]. O índice $\mathcal{M}$ será, então, usado como critério de inferência da estabilidade estática da tensão de sistemas elétricos de potência, via redes neurais, como formulado nesta seção.

4. Rede Neural ARTMAP *Fuzzy*

A rede neural ARTMAP *Fuzzy* [3] é formada por 2 módulos ART *Fuzzy* interligados através de um mecanismo chamado inter-ART, como será descrito adiante. Assim, primeiramente descreve-se o módulo ART *Fuzzy*. Para uma melhor fundamentação operacional das redes neurais ART-descendentes, os vetores usados são representados por linhas e não por colunas, como habitualmente é adotado na literatura especializada.

Os dados de entrada são denotados por vetor $a = [a_1 \ldots a_R]$ R-dimensional, e é normalizado com o intuito de evitar a proliferação de categorias. Assim, $\overline{a} = \dfrac{a}{|a|}$ sendo $\overline{a}$ o vetor de entrada normalizada e $|a| = \sum a_i$.

O vetor de entrada será um vetor $2R$-dimensional, denotado por:

$$E = [\overline{a}\ \overline{a}^c] = [\overline{a}_1 \cdots \overline{a}_R\ \overline{a}_1^c \cdots \overline{a}_R^c]$$

em que $\overline{a}^c = \overline{a}_i^c = 1 - \overline{a}_i^c$.

O vetor de atividade de F_2 é simbolizado por $y = [y_1 \cdots y_S]$ sendo S o número de categorias criadas em F_2. Deste modo, tem-se:

$$y = \begin{cases} 1, & \text{se o nó } J \text{ de } F_2 \text{ é ativo} \\ 0, & \text{caso contrário} \end{cases}$$

Os parâmetros utilizados no processamento da rede ART *fuzzy* são: parâmetro de escolha ($\eta > 0$), taxa de treinamento ($\beta \in [0, 1]$) e parâmetro de vigilância ($\rho \in [0, 1]$). Inicialmente todos os pesos possuem valor igual a 1, indicando assim que não existe nenhuma categoria ativa.

A rede neural ARTMAP [3] é uma rede mais sofisticada, que emprega o treinamento supervisionado, *i.e.*, baseado em estímulos de entrada-saída. É composta por dois módulos ART *fuzzy*; ART_a e ART_b interconectados através de módulo de memória associativa inter-ART, F^{ab}, que possui um mecanismo auto-regulador denominado *match tracking*, que tem a incumbência de maximizar a generalização e minimizar o erro da rede.

O vetor de entrada da rede ART_a é representado pelo vetor $a = [a_1 \ldots a_{R_a}]$, R_a-dimensional, e o vetor de entrada da rede ART_b (que corresponde à saída desejada, referente ao padrão de entrada da rede ART_a) é representado pelo vetor $b = [b_1 \ldots b_{R_b}]$, R_b-dimensional [3].

Os parâmetros usados na rede ARTMAP *fuzzy* são os mesmos usados na rede ART *fuzzy*. A diferença da rede neural ART *fuzzy* é o parâmetro de vigilância do módulo inter-ART, ρ_{ab} ($\rho_{ab} \in [0, 1]$).

A rede ARTMAP realiza o processamento de duas redes ART (ART_a e ART_b), sendo que, após a ressonância ser confirmada em cada rede, tem-se C categoria ativa para a rede ART_a e K categorias ativas para a rede ART_b.

Através do processo *match tracking* é verificado se a categoria ativa no ART_a corresponde ao vetor de saída desejado apresentado no ART_b. O critério de vigilância é dado por [3]:

$$\frac{|y^b \wedge w_{Ck}^{ab}|}{|y^b|} \geq \rho_{ab} \tag{4.1}$$

em que y^b é o vetor saída do ART_b (padrão de atividade F_2^b).

Se (4.1) não for satisfeito, é realizado um incremento mínimo no parâmetro de vigilância ART_a, suficiente apenas para excluir a atual categoria e selecionar outra categoria que se tornará ativa e entrará novamente no processo até que (4.1) seja obedecida. Com a ressonância confirmada, os pesos dos módulos ART_a e ART_b, são atualizados utilizando o mesmo critério da rede neural ART *fuzzy*. A adaptação para o módulo inter-ART é efetuada da seguinte forma: $w_{Ck}^{ab} = 1$ para $k = K$ e $w_{Ck}^{ab} = 0$ para $k \neq K$.

4.1. Estímulo de Entrada da Rede Neural ARTMAP *Fuzzy*

Os vetores padrões de entrada da rede neural são definidos como [4]:

$$X = [P\ Q\ Z]$$

sendo: X o vetor padrão de entrada da rede neural; $P = [P_1\ P_2\ \cdots P_n]$; $Q = [Q_1\ Q_2\ \cdots Q_n]$; P_i a potência ativa da i-ésima barra do sistema; Q_i a potência reativa da i-ésima barra do sistema; Z o vetor contendo as informações em código binário e, n o número de barras do sistema.

Para a realização do treinamento - extração do conhecimento baseado em estímulos de entrada / saída - deve-se proceder a apresentação de um conjunto de dados, no caso, $X = [P\ Q\ Z]$ (entrada) e Y (saída), constituindo um conjunto de pares de treinamento. Trata-se de geração por vetores P e Q, para a geração e para a carga do sistema por um procedimento de distribuição aleatória da geração (despacho aleatório para atender a demanda) e, também, de distribuição aleatória da carga (demanda aleatória) [4]. O vetor Z é formado por informações (em código binário), representando a topologia da rede elétrica, os dados das contingências, etc.

Para se gerar um grande espectro de variação da demanda, o critério a ser usado refere-se à distribuição aleatória da demanda e, conseqüentemente, da geração para atender a demanda, tomando-se variações percentuais da carga e da geração em torno do caso base (considerando como sendo o perfil de geração / carga de 100%).

4.2. Estímulo de Saída da Rede Neural ARTMAP *Fuzzy*

Os estímulos de saída compreendem, neste artigo, os valores da margem de segurança ($\mathcal{M}$), calculados conforme descrito na Seção 3. Ou seja, os padrões de treinamento correspondem aos parâmetros: $X_j = [P_j\ \ Q_j\ \ Z_j]$ (entradas); $Y_j = [\underline{\mathcal{M}_\mathrm{j}}]$ (saidas); $j = 1,\ 2,\ \cdots,\ np$, sendo $\underline{\mathcal{M}_\mathrm{j}}$ a $j-$ésima margem de segurança representada por classes de severidade; np o número de pares de padrões para a fase de treinamento.

A saída $\underline{\mathcal{M}_\mathrm{j}}$ é codificada por classes (em código binário), sendo que cada classe expressa o grau de severidade.

5. Aplicação

Apresentam-se, a seguir, os resultados obtidos através do uso do método proposto, considerando-se um sistema composto por 45 barras, 72 linhas de transmissão e 10 máquinas síncronas. Os dados das máquinas síncronas e do sistema de transmissão pode ser visto na Figura 2.

O treinamento da rede neural foi realizado considerando-se um conjunto de 700 perfis de geração / carga e respectivas margens de segurança $\mathcal{M}$. Cada perfil corresponde a um redespacho de geração em relação ao caso base, realizado de forma aleatória para atender a demanda, também fixada de modo aleatório em cada barra. O universo de variação de carga encontra-se correspondido entre 65 e 135%, em relação à carga total do sistema. Portanto, cada perfil é gerado considerando-se um percentual de variação em torno do estado nominal (caso base) e determinada semente para o processo de geração de seqüências aleatórias. Assim, para um mesmo percentual, diferentes sementes poderão gerar diferentes despachos de geração de diferentes perfis de carga. Realizado o treinamento da rede neural, os testes (análise da estabilidade de tensão) podem ser efetivados, cujos resultados são mostrados na Tabela 1. Os resultados da análise são apresentados na forma de classes de 1 a 10.

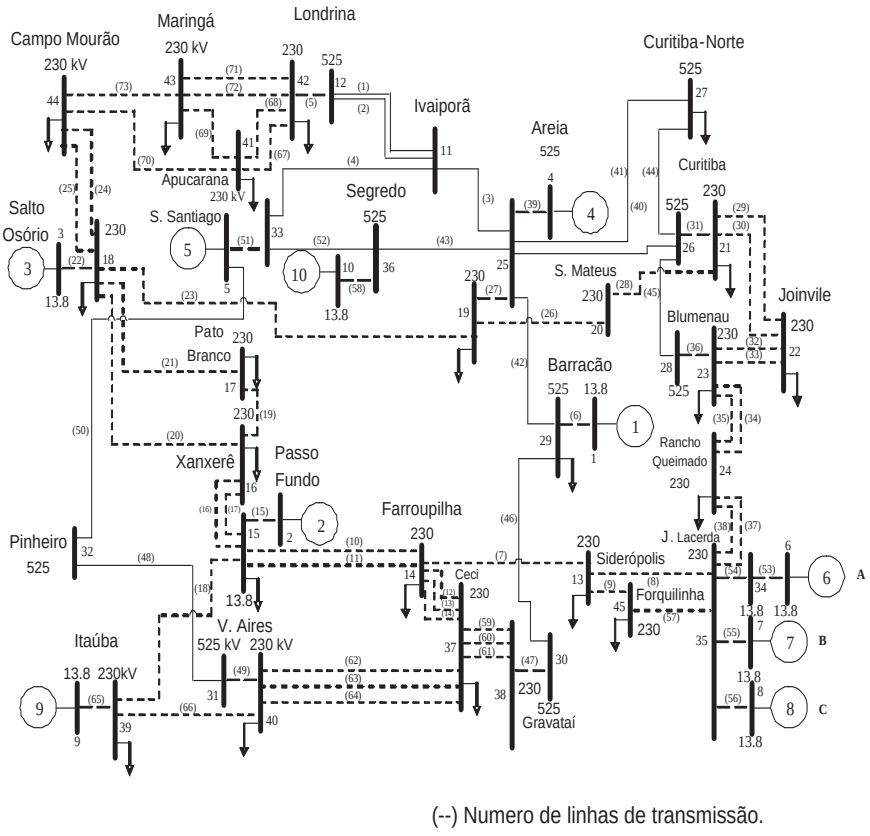


Figura 2: Representação do sistema teste.

Estas classes são adotadas visando constituir as saídas em código binário, *i.e.*, um modo mais adequado para se usar as redes neurais da família ART. Assim, adotou-se a seguinte definição das classes $1, 2, \ldots, 10$ (representação com 4 *bits*) correspondem a $0 \leq \mathcal{M} \leq 0.1,\ 0.1 \leq \mathcal{M} \leq 0.2,\ \ldots,\ 0,9 \leq \mathcal{M} \leq 1.0$, respectivamente. Com estas definições, pode-se inferir sobre as "distâncias" que cada perfil de geração / carga se encontra em relação à fronteira da estabilidade de tensão. Tais intervalos, associados às classes, podem ser definidos, aumentando-os ou diminuindo-os de acordo com o interesse do usuário. Neste caso, devendo-se ajustar o número de *bits* em função da maior ou da menor discretização usada. Ressalta-se que os acertos do sistema neural são superiores a 85%, considerando-se 700 simulações realizadas. Este percentual, certamente irá aumentar, à medida em que novos padrões forem incorporados à base de dados, através do processo referente ao treinamento continuado.

Tabela 1: Comparação dos métodos.

		Margem $\mathcal{M}$				Margem $\mathcal{M}$	
%	Semente	*Simul*	Artmap Fuzzy	%	Semente	*Simul*	Artmap Fuzzy
-32.5	123	.5994	6	17.5	123	.6722	7
-27.5	123	.6354	7	17.5	6691	.6183	7
-22.5	614	.7096	8	22.5	145	.6055	6
-12.5	31	.6551	8	22.5	221	.6528	7
-7.5	123	.7003	7	22.5	377	.5662	6
-2.5	7116	.7134	8	27.5	7116	.4349	4
7.5	31	.7089	8	27.5	8555	.5840	6
12.5	74	.6694	7	32.5	123	.5873	6

6. Conclusão

Foi apresentada uma nova metodologia para análise de estabilidade de tensão estática de sitemas elétricos de potência multinodal. Trata-se de uma metodologia usando uma rede neural ARTMAP *fuzzy*, cujo treinamento é realizado a partir de uma base de dados gerada via simulação (realizada usando o programa computacional *Simul* [4]); cálculo de fluxo de potência, bem como outras grandezas que foram implementadas visando atender a necessidade do estudo proposto. Tendo em vista que a estratégia de obtenção desta base de dados (definição de perfil de geração e de carga de modo aleatório) permitiu estabelecer um espectro de variação de modos operativos do sistema que "simule"estados operativos reais, a arquitetura ARTMAP *fuzzy*, tal como as demais redes da família ART, apresenta a característica de plasticidade que admite implementar o treinamento continuado, o que se diferencia, de forma vantajosa, em relação às tradicionais redes neurais comumente usadas na literatura especializada.

Abstract. This work presents a new methodology for the analysis of the static voltage stability in electric power systems. It is a neural inference system based on *fuzzy* ARTMAP architecture. This system presents accurate results with very fast response, being able to realize real time analysis. Besides, it allows the users work with flexibility where structural modifications are required (real situation in system operation), when compared to tradicional neural networks used in specialized literature. To illustrate the proposed neural network an apllication is presented considering an eletrical power system composed by 45 buses, 72 transmission line and 10 syncrhonous machines.

Referências

[1] L.D. Arya, S.C. Choube, M. Shrivastava, Technique for voltage stability assessment using newly developed line voltage stabiblity index, *Energy Conversion and Management*, **49**, No. 2 (2008), 267-275.

[2] S. Barnett, C. Storey, "Matrix Methods in Stability Theory", Thomas Nelson and Sons Ltd, London, 1970.

[3] G.A. Carpenter, S. Grossberg, N. Markuzon, J.H. Reynolds, D.B. Rosen, Fuzzy ARTMAP: A neural network architecture for incremental supervised learning of analog multidimentional maps, *IEEE Transactions on Neural Networks*, **3**, No. 5 (1992), 698-713.

[4] W.P. Ferreira, M.C.G. Silveira, A.D.P. Lotufo, C.R. Minussi, Transient stability analysis of electric energy system via a Fuzzy ART-ARTMAP neural network, *Electric Power System Research*, **76** (1996), 475-484.

[5] J.C. Geromel, "Methods and Techniques For Decentralized Control Systems Analysis and Design", Cooperativa Libraria Universitaria del Politecnico, Milano, 1987.

[6] S. Haykin, "Neural Networks: A Comprehensive Foundation", Prentice-Hall, New Jersey, 1994.

[7] C.-T Su, F.C. Chang, Quick evaluation of determinants, *Applied Mathematics and Computation*, **75** (1996), 117-118.

[8] A. Tiranuchit, R.J. Thomas, A posturing strategy against voltage instabilities in electric power systems, *IEEE Transaction on Power Systems*, **3**, No. 1 (1988), 87-93.

Available online at www.sciencedirect.com
ScienceDirect
Electric Power Systems Research 77 (2007) 730–738
ELECTRIC POWER SYSTEMS RESEARCH
www.elsevier.com/locate/epsr

Sensitivity analysis by neural networks applied to power systems transient stability

Anna Diva P. Lotufo *, Mara Lúcia M. Lopes, Carlos R. Minussi

Departamento de Engenharia Elétrica, UNESP, Campus de Ilha Solteira, Av. Brasil, 56, 15385-000 Ilha Solteira, SP, Brazil

Received 29 March 2005; received in revised form 26 August 2005; accepted 1 September 2005
Available online 31 July 2006

Abstract

This work presents a procedure for transient stability analysis and preventive control of electric power systems, which is formulated by a multilayer feedforward neural network. The neural network training is realized by using the back-propagation algorithm with fuzzy controller and adaptation of the inclination and translation parameters of the nonlinear function. These procedures provide a faster convergence and more precise results, if compared to the traditional back-propagation algorithm. The adaptation of the training rate is effectuated by using the information of the global error and global error variation. After finishing the training, the neural network is capable of estimating the security margin and the sensitivity analysis. Considering this information, it is possible to develop a method for the realization of the security correction (preventive control) for levels considered appropriate to the system, based on generation reallocation and load shedding. An application for a multimachine power system is presented to illustrate the proposed methodology.
© 2006 Elsevier B.V. All rights reserved.

Keywords: Sensitivity analysis; Preventive control; Transient stability; Neural networks; Back-propagation

1. Introduction

Sensitivity analysis [1] is a very important tool to solve several problems present in many areas of human knowledge: engineering, mathematics, physics, economy, medicine, biology, etc., especially when nonlinearities are involved. Therefore, it is possible to infer about the behavior of the system face to parametric variations without the need of solving a problem that involves great complexity, described by a set of nonlinear differential and algebraic equations. The comportamental conclusions are extracted from the calculation of the derivative function under analysis. This is the problem studied in this work, analysis of transient stability for electric energy systems and specifically the preventive control problem [2].

Transient stability analysis is one of the main studies used in electric power systems (EPS). It is a procedure to evaluate the effects caused by perturbations which origin great deviations on the angles of the synchronous machines, e.g., short-circuit, outage/input of electric equipment. In this case, the model of the system is described by a set of nonlinear algebraic and differential equations [3]. Due to unstable cases and/or equipment capability violations, it is necessary to adopt procedures that can lead the system to a secure state, known as security control. The methods for dynamical preventive control have been proposed recently and the publication available in the literature is not abundant, e.g., [2–6].

This work presents a methodology based on neural networks [7–9] to analyze the transient stability – considering short-circuit faults with transmission line outages – and, principally to provide the sensitivity analysis of EPS, that represent the necessary instruments for implementing the preventive control. Neural networks are important resources to deal with the preventive control problem, due to the training be an off-line activity and the analysis be concluded with minimal computational effort (basically the calculus with the input and output of the neural network), being useful for applications in real time. It is important to emphasize that the sensitivity calculus is carried out without computational effort. On the other hand, to obtain the sensitivity model by conventional procedures, require a large quantity of complex matrix calculation, spending much time, principally for applications in huge systems.

* Corresponding author. Tel.: +55 18 37431222; fax: +55 18 37431163.
E-mail addresses: annadiva@dee.feis.unesp.br (A.D.P. Lotufo), mara@dee.feis.unesp.br (M.L.M. Lopes), minussi@dee.feis.unesp.br (C.R. Minussi).

0378-7796/$ – see front matter © 2006 Elsevier B.V. All rights reserved.
doi:10.1016/j.epsr.2005.09.020

The neural network used is a feedforward multilayer with training by back-propagation (BP) algorithm [10]. The BP algorithm training rate is adjusted by a fuzzy controller [11–14], which monitories the global error and the global error variation during the training as well the adaptive process [15,16]. It is an optimal mechanism that reduces the convergence time and improves the precision of the results, as observed in ref. [15]. The variables used on the training are causal variables of a problem of transient stability analysis (active and reactive nodal electric power which are the input neural network stimulus) and the security margins (output neural network stimulus) generated using the potential energy boundary surface (PEBS) iterative method [17], microcomputer version. The security margin expressed in function of total energy can be interpreted as being a measure of the distance in relation to the condition of the instability of the system. The sensitivity model is referred to the relation with the security margin and the nodal electric power. Thus, the generation reallocation and load shedding necessary for obtaining a secure state of the system can be evaluated, that is, a security level considered adequate for transient stability. It is a tool that can either aid the prevention or, at least, minimize the effects of a blackout [18–20]. An application considering a multimachine system is presented for testing the proposed methodology.

2. System model

Considering an electrical power system composed of *ns* synchronous machines, the dynamical behavior of the *i*th machine, related to Center of Angles (CA), is described by the following differential equation (classical model) [12,21]:

$$M_i\ddot{\theta}_i - P_i(\theta) = 0, \quad i \in N \tag{1}$$

where

$$P_i(\theta) = \mathrm{Pm}_i - \mathrm{Pe}_i - \frac{M_i \mathrm{PCOA}}{MT}; \tag{2}$$

M_i is $2H_i/\omega_s$; $\omega_s \triangleq$ synchronous speed of the rotor, $2\pi f_0$; H_i is the inertia constant (s); f_0 is the nominal frequency of system (Hz); $\theta_i \triangleq$ rotor angle of *i*th synchronous machine related to CA (electrical radians), $\delta_i - \delta_0$; δ_i is the rotor angle of *i*th synchronous machine in relation to synchronously rotating reference frame (in electrical radians); δ_0 is $\sum_{j \in N} M_j\delta_j$; Pm_i is the mechanical power of input (pu); Pe_i is the electrical power of output (pu); PCOA $\triangleq$ accelerating power of CA, $\sum_{j \in N}(\mathrm{Pm}_j - \mathrm{Pe}_j)$; MT is $\sum_{j \in N} M_j$; $N \triangleq$ index set of synchronous machines that comprises the system, $\{1, 2, \ldots, ns\}$; *ns* is the number of electrical synchronous machines.

3. Transient stability analysis

The transient stability analysis of EPS, considering a contingency of index *r*, is effectuated using the security margin criterion [12,17,21]:

$$\mathcal{M}_r = \frac{E_{\mathrm{crit}_r} - E_{\mathrm{e}_r}}{E_{\mathrm{crit}_r}} \tag{3}$$

where E_{crit_r} is the total critical energy of the system; E_{e_r} is the total energy of the system evaluated on the instant of the fault elimination (te).

The critical energy (E_{crit}) and the critical time (t_{crit}), is determined by the iterative PEBS method [4,16], or another procedure that presents a similar result, principally in relation to precision. The total energy related to system (1), is given by [12,17,21]:

$$E(\theta, \omega) = E_{\mathrm{c}}(\omega) + E_{\mathrm{p}}(\theta) \tag{4}$$

where

$$E_{\mathrm{c}}(\omega) = \text{kinetic energy} = \frac{1}{2}\sum_{i \in N} M_i w_i^2; \tag{5}$$

$$E_{\mathrm{p}}(\theta) = \text{potential energy} = -\sum_{i \in N} \int_{\theta_i^p}^{\theta_i} P_i(\theta)\, \mathrm{d}\theta_i. \tag{6}$$

Then, the transient stability for the *r*th contingency is evaluated by the security margin on the following way [17]:

- if $\mathcal{M}_r \geq 0$, the system is considered *stable*, for transient stability;
- if $\mathcal{M}_r < 0$, the system is considered *unstable*, for transient stability.

4. Dynamic preventive control

Considering a list composed of *S* contingencies, the security margin of the system must satisfy the following relation [4]:

$$\mathcal{M} \geq M_{\min} \tag{7}$$

where $M_{\min}$ is the minimum limit of the security margin of the system ($M_{\min} > 0$); $\mathcal{M} \triangleq \min(\mathcal{M}_r, r = 1, 2, \ldots, S)$.

The control actions must cause modifications on the security margins and the following relations must be satisfied [19,4]:

$$\mathcal{M}_r = (\mathcal{M}_r^0 + \Delta\mathcal{M}_r) \geq M_{\min}, \quad r = 1, 2, \ldots, S \tag{8}$$

where $\mathcal{M}_r$ is the security margin referred to the *r*th contingency.

The necessary changing ($\Delta\mathcal{M}_r$) to correct the security margin – in terms of a vector $\boldsymbol{X}$ – is estimated by the sensitivity theory, of first order, according to [19]:

$$\Delta\mathcal{M}_r \cong \left[\frac{\partial\mathcal{M}_r}{\partial \boldsymbol{X}}\right]^{\mathrm{T}} \Delta\boldsymbol{X} \tag{9}$$

or

$$\Delta t_{\mathrm{crit}_r} \cong \left[\frac{\partial t_{\mathrm{crit}_r}}{\partial \boldsymbol{X}}\right]^{\mathrm{T}} \Delta\boldsymbol{X} \tag{10}$$

where $\partial\mathcal{M}_r/\partial\boldsymbol{X}$ is the sensitivity of the security margin in relation to the vector $\boldsymbol{X}$; $\partial t_{\mathrm{crit}}/\partial\boldsymbol{X}$ is the sensitivity of the critical time in relation to the vector $\boldsymbol{X}$; $\Delta\boldsymbol{X}$ is the vector corresponding to the changing on the components of vector $\boldsymbol{X}$.

Equations (9) and (10) represent the sensitivity models considering the interest variables: security margin and critical time, respectively.

The vector $\boldsymbol{X}$, in this work, is represented by the nodal active power. The sensitivity $\partial \mathcal{M}_r/\partial \boldsymbol{X}$ is developed in Section 6 by neural networks.

5. Neural network structure

This section presents a multilayer feedforward neural network, whose training is realized by a back-propagation algorithm, which is a systematic procedure of adaptation of the neural network weights based on the descendent gradient [15].

The ith output element (neuron) [6] is a linear combination of the element inputs x_j that are connected to the element i by the weight w_{ij}:

$$\vartheta_i = \sum_j w_{ij} x_j \tag{11}$$

Each element can have a bias w_0 plus an extra constant input $x_0 = +1$. The linear output ϑ_i is finally converted in a nonlinear function, as a sigmoid, relay [8,10], etc. The relay functions are appropriated for binary systems, while the sigmoid functions can be employed for both continuous and binary systems.

6. Neural network training using the adaptive process

The back-propagation algorithm (BP) is realized on the following way:

(1) Present to the network a pattern $\boldsymbol{X}$, that gives an output $\boldsymbol{Y}$.
(2) Calculus of the error (difference between the desired value and the output) for each output.
(3) Determination of the back-propagated error by the network associated to the partial derivative of the quadratic error of each element related to the weights.
(4) Adjustment of the weights of each element.
(5) A new pattern is presented to the network and the process is repeated until the convergence, i.e. (|error| ≤ predefined tolerance).

Random numbers is adopted for the initial weights, considering the interval {0,1} [10]. The back-propagation BP algorithm consists of adapting the weights in such way that the quadratic error be minimized. The summation of the instantaneous quadratic error of each neuron of the last layer (network output) is given by:

$$\varepsilon^2 = \sum_{i=1}^{no} \varepsilon_i^2 \tag{12}$$

where ε_i is the $d_i - y_i$; d_i is the desired output of the ith element of the last layer; y_i is the output of the ith element of the last layer; no is the quantity of neurons of the last layer.

Considering the ith neuron of the network and using the descent gradient method [10], the adjustment of the weights is formulated by [10]:

$$\boldsymbol{V}_i(r+1) = \boldsymbol{V}_i(r) + \theta_i(r) \tag{13}$$

where $\theta_i(r)$ is $-\gamma[\nabla_i(r)]$; γ is the control of the stability parameter or training rate; $\nabla_i(r)$ is the gradient of the quadratic error related to the weights of the neuron i; $\boldsymbol{V}_i \triangleq$ vector containing the weights of the neuron i, $[w_{0i}\ w_{1i}\ w_{2i} \ldots w_{ni}]^{\mathrm{T}}$.

The adopted direction in ref. (13) for minimizing the objective function of the quadratic error corresponds to the opposite direction of the gradient. The parameter γ determines the size of the vector $\theta_i(r)$. The nonlinear function used is the sigmoid function defined by [7,10]:

$$y_i = \frac{1 - \mathrm{e}^{(-\lambda\vartheta_i + \rho)}}{1 + \mathrm{e}^{(-\lambda\vartheta_i + \rho)}} \quad \text{for } y_i \in [-1, +1] \tag{14}$$

or

$$y_i = \frac{1}{1 + \mathrm{e}^{(-\lambda\vartheta_i + \rho)}} \quad \text{for } y_i \in [0, +1] \tag{15}$$

where λ is the constant that determines the inclination of the function y_i; ρ is the constant that determines the translation of the function y_i.

The gradient $\nabla_i(r)$ is represented by:

$$\nabla_i(r) = \frac{\partial \varepsilon_i^2}{\partial V_i(r)} = 2\varepsilon_i \frac{\partial \varepsilon_i}{\partial V_i(r)} \tag{16}$$

Differentiating the error in relation to the vector V_i, follows:

$$\frac{\partial \varepsilon}{\partial V_i} = -\frac{\partial y_i}{\partial V_i} = \frac{\partial y_i}{\partial \vartheta_i}\frac{\partial \vartheta_i}{\partial V_i} = \sigma_i \frac{\partial \vartheta_i}{\partial V_i} \tag{17}$$

where $\sigma_i \triangleq$ derivative of the sigmoid function, related to ϑ_i;

$$= \frac{1}{2}\lambda(1 - y_i^2), \tag{18}$$

for sigmoid function in Eq. (14);

$$= \lambda y_i(1 - y_i), \tag{19}$$

for sigmoid function in Eq. (15).

Then

$$\frac{\partial \vartheta_i}{\partial V_i} = \boldsymbol{X}_i \tag{20}$$

where $\boldsymbol{X}_i \triangleq$ pattern vector, $[x_{0i}\ x_{1i}\ x_{2i} \ldots x_{ni}]^{\mathrm{T}}$.

Therefore, by using the gradient descent method, the following schema of the adaptation of the weights is obtained [15,22,10]:

$$\boldsymbol{V}_i(r+1) = \boldsymbol{V}_i(r) + 2\gamma\beta_i \boldsymbol{X}_i \tag{21}$$

If the ith element is in the last layer, then:

$$\beta_i = \sigma_i \varepsilon_i \tag{22}$$

If the ith element is in the other layers, then:

$$\beta_i = \sigma_i \sum_{j \in Q(i)} w_{ij}\beta_j \tag{23}$$

where $Q(i)$ is the set of the index that are in the next layer to the layer of the ith element and are interconnected to the ith element.

The parameter γ used as a stability control of the iterative process is dependent on λ [22]. For convenience, the parameter γ (training rate) is redefined as follows [13]:

$$\gamma = \frac{\gamma^*}{\lambda} \tag{24}$$

Substituting (24) in (21), the dependence on the amplitude σ related to λ is canceled. The amplitude σ is maintained constant for every λ. This alternative is important, because λ is only acting on the left and right tails of σ. Then (21) is written as follows [13]:

$$\boldsymbol{V}_i(r+1) = \boldsymbol{V}_i(r) + \left\{ 2\gamma^* \frac{\beta_i}{\lambda} \right\} \boldsymbol{X}_i \tag{25}$$

The BP algorithm is considered on the technical literature as a benchmark in precision, although the convergence is very slow. Therefore, it is proposed to adjust the convergence rate γ^* during the convergence process aiming at reducing the execution training time. The adjustment of γ^* is executed by the procedure based on the fuzzy controller.

Similar to the procedure of the adjustment of the weights effectuated by the descent gradient method, the adaptation of the inclination and translation parameters of the postsynaptic function is realized.

The general form of the postsynaptic functions used for adjusting the neural network is given by [16]:

$$y_i = f(\lambda, \rho, \vartheta, w) \tag{26}$$

The inclination and translation parameters contain learning rates denoted by γ^λ and γ^ρ, respectively.

Fig. 1 illustrates the neural architecture with the adjustments of the inclination and translation parameters of the postsynaptic function, i.e., the sigmoid function [16].

The adjustment of the inclination parameter of the sigmoid function is given by [15]:

$$\boldsymbol{\lambda}_i(r+1) = \boldsymbol{\lambda}_i(r) + \theta_i^\lambda(r) \tag{27}$$

where $\theta_i^\lambda(r)$ is $-\gamma^\lambda[\nabla_i^\lambda(r)]$; $\nabla_i^\lambda(r)$ is the gradient of the quadratic error related to the inclination of the neuron i; $\boldsymbol{\lambda}_i \triangleq$ vector containing the inclinations of the neuron i, $[\lambda_{0i}\ \lambda_{1i}\ \lambda_{2i} \ldots \lambda_{ni}]^{\mathrm{T}}$.

The gradient ∇_i^λ is represented by [15]:

$$\nabla_i^\lambda(r) = \frac{\partial \varepsilon_i^2}{\partial \lambda_i(r)} = 2\varepsilon_i \frac{\partial \varepsilon_i}{\partial \lambda_i} = -2\varepsilon_i \frac{\partial y_i}{\partial \lambda_i} = -2\varepsilon_i \sigma_i^\lambda \tag{28}$$

where $\sigma_i^\lambda \triangleq$ derivative of the sigmoid function, related to λ_i.

The general rule that defines the adaptation of the inclination parameter of the sigmoid function is given by the following way:

$$\lambda_i(r+1) = \lambda_i(r) + 2\gamma^\lambda \beta_i^\lambda \tag{29}$$

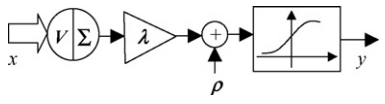


Fig. 1. Neural architecture for adjusting the inclination and translation parameters of the postsynaptic function.

The rule that defines the calculus of β_i^λ follows the next two Eqs. (30) and (31), according to:

(a) if it is in the last layer:

$$\beta_i^\lambda = \sigma_i^\lambda \varepsilon_i \tag{30}$$

(b) if it is in the other layers:

$$\beta_i^\lambda = \sigma_i^\lambda \sum_{j \in Q(i)} w_{ij} \beta_j^\lambda \tag{31}$$

Similar to the procedure described above, the adjustment of the translation parameter of the sigmoid function is given by:

$$\boldsymbol{\rho}_i(r+1) = \boldsymbol{\rho}_i(r) + \theta_i^\rho(r) \tag{32}$$

where $\theta_i^\rho(r)$ is $-\gamma^\rho[\nabla_i^\rho(r)]$; $\nabla_i^\rho(r)$ is the gradient of the quadratic error related to the displacement of the neuron i; $\boldsymbol{\rho}_i \triangleq$ vector containing the displacements of the neuron i, $[\rho_{0i}\ \rho_{1i}\ \rho_{2i} \ldots \rho_{ni}]^{\mathrm{T}}$.

The gradient ∇_i^ρ is calculated on the following way:

$$\nabla_i^\rho(r) = \frac{\partial \varepsilon_i^2}{\partial \rho_i(r)} = 2\varepsilon_i \frac{\partial \varepsilon_i}{\partial \rho_i} = -2\varepsilon_i \frac{\partial y_i}{\partial \rho_i} = -2\varepsilon_i \sigma_i^\rho \tag{33}$$

where $\sigma_i^\rho \triangleq$ derivative of the sigmoid function, related to ρ_i.

The adaptation rule of the translation parameter of the sigmoid function is shown by equation [15]:

$$\rho_i(r+1) = \rho_i(r) + 2\gamma^\rho \beta_i^\rho \tag{34}$$

The rule that defines the calculus of β_i^ρ follows the Eqs. (35) and (36), according to:

(a) if it is in the last layer:

$$\beta_i^\rho = \sigma_i^\rho \varepsilon_i \tag{35}$$

(b) if it is in the other layers:

$$\beta_i^\rho = \sigma_i^\rho \sum_{j \in Q(i)} w_{ij} \beta_j^\rho \tag{36}$$

The adaptation rule of the inclination and translation parameters of the network is calculated in an iterative form for every ith neuron. Therefore, the adaptive neural network always gives an output and the network is faster than the conventional ones.

7. Sensitivity analysis by neural networks

Supplying a pattern $\boldsymbol{X} \in \Re^n$ to the network, which gives an output $\boldsymbol{Y} \in \Re^m$ initializes the BP algorithm. In the sequence, an error in each output (the difference between the desired value and the output) is calculated. Next step is to determine the error propagated in inverse way by the network associated to the partial derivative of the quadratic error of each element related to the weights, and finally to adjust the weights in each element. Then, a new pattern is presented, and the pattern must be repeated until convergence ($|\text{error}| \leq$ predefined tolerance).

After concluding this step, the training mechanism does not actuate, including the fuzzy controller. Thus, the network is able to generalize, i.e., applying any input pattern vector, propagating the signal on the straight sense (input to output), an evaluation of the analysis (diagnosis) results on the output, providing a mapping, $\boldsymbol{X} \rightarrow \boldsymbol{Y} = \boldsymbol{f}(\boldsymbol{X})$, $\boldsymbol{X} \in \Re^n$ and $\boldsymbol{Y} \in \Re^m$.

The derivatives of the output variables (sensitivity analysis) in relation to the input vector components are estimated using a neural network structure trained according to the following description. The sensitivity analysis, by neural networks, is used in this work to obtain $\partial \mathcal{M}_r / \partial \boldsymbol{X}$ (problem defined by Eq. (9)). For the solution of the preventive control problem, the generation reallocation and load shedding (according to the formulation described in Section 4) is adopted where the vector $\boldsymbol{X}$ corresponds to the nodal active power ($\boldsymbol{P}$).

Thus, consider $\boldsymbol{X}^k$ and $\boldsymbol{Y}^k$ as being the kth pair of input and output vector of the neural network. The feedforward neural network shown in Fig. 2 is also considered. It is the representation of a network composed of three layers, where the variables on the principal points of the network and the weight matrices are defined. The input and output layers have n and m neurons, respectively, where n is the dimension of input vector $\boldsymbol{X}^k$ and m is the dimension of output vector $\boldsymbol{Y}^k$.

The partial derivative of y_p^k (pth component of vector $\boldsymbol{Y}^k$) in relation to x_j^k (jth component of the input vector $\boldsymbol{X}^k$) is desired to obtain. Then, to obtain these partial derivatives, is necessary to obtain the intermediate partial derivatives (on the output of the neurons of the hidden layers) of the neural network. Thus, the calculus of partial derivatives of z_i^k (ith component of output vector $\boldsymbol{z}^k$), in relation to x_j^k, is obtained in the following way [8]:

$$\frac{\partial z_i^k}{\partial x_j^k} = w_i^k \frac{\lambda}{2}(1 - \{z_i^k\}^2) \tag{37}$$

for sigmoid function (14), or

$$\frac{\partial z_i^k}{\partial x_j^k} = w_i^k \lambda z_i^k (1 - z_i^k) \tag{38}$$

for sigmoid function (15), where x_j^k is the jth component of input vector $\boldsymbol{X}^k$; $\boldsymbol{X}^k = [x_1^k\, x_2^k \ldots x_n^k]^{\mathrm{T}}$; k is the index referred to the kth pattern vector.

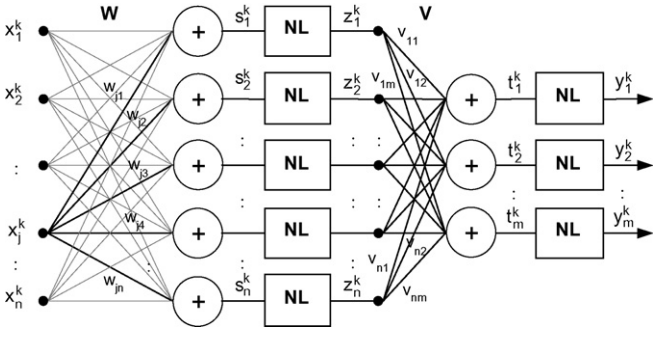


Fig. 2. Feedforward neural network.

The pth intermediate output (input of the sigmoid function) of the output layer of the neural network is expressed by:

$$t_p^k = \{\boldsymbol{z}^k\}^{\mathrm{T}} \boldsymbol{v}_p, \qquad p = 1, 2, \ldots, m \tag{39}$$

where $\boldsymbol{z}^k = [z_1^k\, z_2^k \ldots z_n^k]^{\mathrm{T}}$; $\boldsymbol{v}_p = [v_{1p}\, v_{2p} \ldots v_{np}]^{\mathrm{T}}$.

Thus,

$$\frac{\partial y_p^k}{\partial x_j^k} = \left\{ \frac{\partial y_p^k(\lambda, t_p^k)}{\partial t_p^k} \right\} \frac{\partial t_p^k}{\partial x_j^k} \tag{40}$$

as

$$\frac{\partial t_p^k}{\partial x_j^k} = \{\boldsymbol{v}_p^k\}^{\mathrm{T}} \frac{\partial \boldsymbol{z}^k}{\partial x_j^k} \tag{41}$$

where

$$\frac{\partial \boldsymbol{z}^k}{\partial x_j^k} = \left[\frac{\partial z_1^k}{\partial x_j^k} \frac{\partial z_2^k}{\partial x_j^k} \cdots \frac{\partial z_n^k}{\partial x_j^k} \right]^{\mathrm{T}} \tag{42}$$

Then, substituting Eqs. (14) or (15) in Eq. (40), follows:

$$\frac{\partial y_p^k}{\partial x_j^k} = \frac{\lambda^2}{4(1 - \{y_p^k\}^2)} \{\boldsymbol{v}_p\}^T \boldsymbol{b}_j^k \tag{43}$$

for sigmoid function (14), or

$$\frac{\partial y_p^k}{\partial x_j^k} = \lambda^2 y_p^k (1 - y_p^k) \{\boldsymbol{v}_p\}^{\mathrm{T}} \boldsymbol{b}_j^k \tag{44}$$

for sigmoid function (15), where

$$\boldsymbol{b}_j^k = [(1 - \{z_1^k\}^2) w_{j1} (1 - \{z_2^k\}^2) w_{j2} \ldots (1 - \{z_n^k\}^2) w_{jn}]^{\mathrm{T}} \tag{45}$$

for function (14), or

$$\boldsymbol{b}_j^k = [z_1^k (1 - z_1^k) w_{j1}\, z_{2k} (1 - z_1^k) w_{j2} \ldots z_n^k (1 - z_n^k) w_{jn}]^{\mathrm{T}} \tag{46}$$

for function (15).

8. Application

The application is conducted by using a system composed of 10 synchronous machines, 73 transmission lines and 45 busses, based on a configuration of the South Brazilian System, whose online diagram is shown in Fig. 3, and the other data are in ref. [4].

Three-phase short-circuits eliminated in 0.15 s (9 cycles for a 60 Hz operation), by tripping a transmission line are analyzed.

The neural network training was effectuated considering a set of 50 generation and load profiles and respective security margin. Each profile corresponds to a generation reallocation in relation to a base case in a random way to attend the demand, also fixed in a random way in each bus. The universe of the load variation is between 95% and 112.5%, in relation to the nominal load of the system. Therefore, each profile is generated considering a variation percentile around the nominal state (base

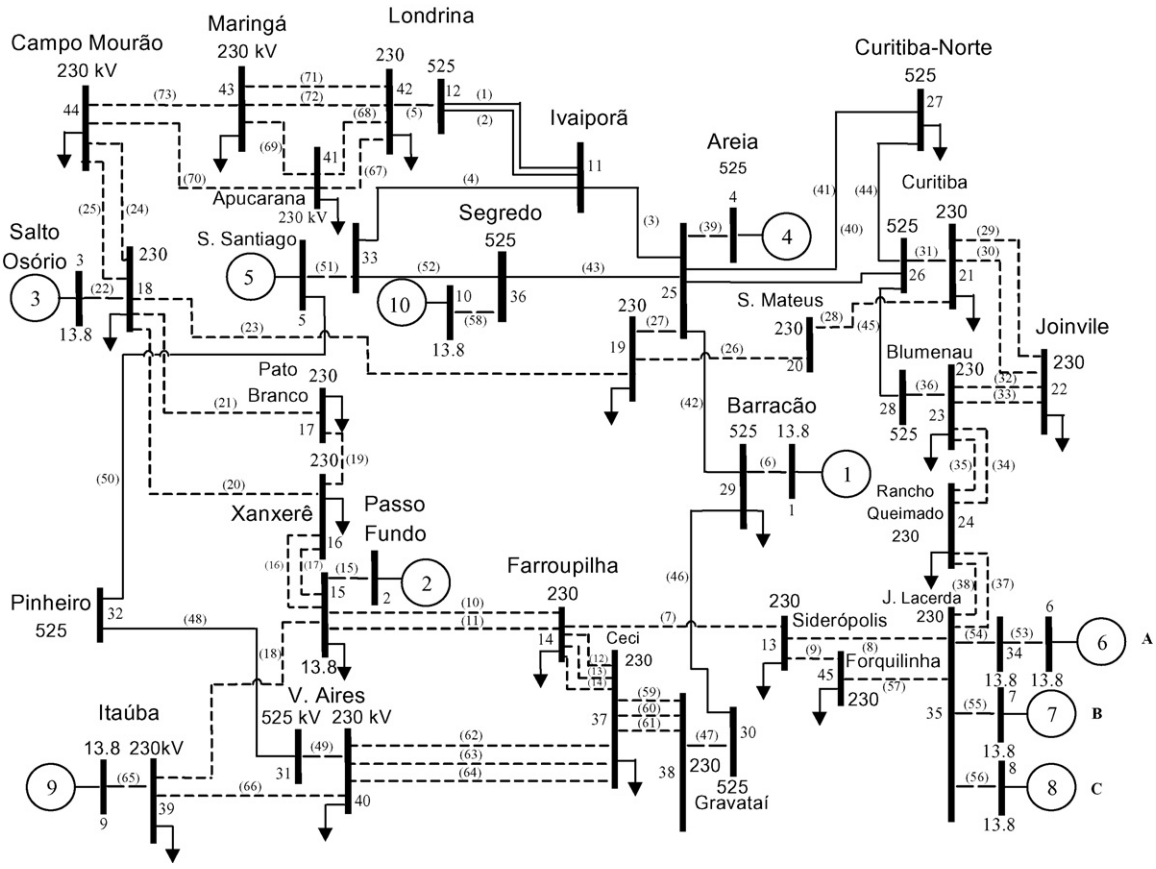


Fig. 3. South-Brazilian System used on the simulations.

case—100%) and a respective random number to the random sequence generation process. Thus, to a same percentile, different generation random numbers generate different generation dispatches of different load profiles. This proceeding generates an adequate set of patterns to the training phase. The results are shown in Table 1.

Table 2 shows the results of the transient stability analysis for 15 contingencies randomly selected. The generation/load profile is chosen in a pseudo random form (112.5% of the base case and random number equal to 2520). It represents a possible level of generation to attend a determined demand. It is a hypothetical

Table 1
Results of the transient stability analysis for the initial generation and load profile (base case—100%)

Contingency number	Data of the defect		Security margin ($\mathcal{M}$)
	Bus under short-circuit	Transmission line removed	
1	17	16–17	0.8470
2	15	15–16	0.8250
3	36	25–36	0.9161
4	29	25–29	0.8056
5	15	14–15	0.8416
6	33	11–33	0.8630
7	29	29–30	0.8481
8	16	16–17	0.8447
9	16	16–18	0.8300
10	25	25–27	0.9440
11	30	29–30	0.9800
12	39	39–40	0.2089
13	13	13–45	0.9320
14	25	25–29	0.9503
15	25	25–26	0.9444

Table 2
Results of the transient stability analysis considering a generation and load profile corresponding to a pseudo-random distribution of 12.5% in relation to the base case and a random number equal to 2520

Contingency number	Data of the defect		Security margin ($\mathcal{M}$)
	Bus under short-circuit	Transmission line removed	
1	17	16–17	0.8222
2	15	15–16	0.7262
3	36	25–36	0.9016
4	29	25–29	0.7824
5	15	14–15	0.7494
6	33	11–33	0.8829
7	29	29–30	0.8704
8	16	16–17	0.8311
9	16	16–18	0.8123
10	25	25–27	0.9449
11	30	29–30	0.9750
12	39	39–40	−0.8188
13	13	13–45	0.4829
14	25	25–29	0.9421
15	25	25–26	0.9486

operation profile, where is realized the security evaluation of the system, and in this work consists of the transient stability analysis considering a group of contingencies.

Contingence 12 is the most critical (least security margin $\mathcal{M}_{12}^0 = -0.8188$). Taking into account the objective of this work that is the generation of the sensitivity model (by neural networks) with the security margin and the power vector (generation and load), i.e., the relation with these two variables is defined. The precision of this sensitivity model can be verified executing variations, for example, on the generation profile of the system and "measure" the variation produced on the security margin.

Therefore, to correct the security margin associated to this contingence, and meet the restriction defined by Eq. (8): $\mathcal{M}_{12} = \mathcal{M}_{12}^0 + \Delta\mathcal{M}_{12} \geq M_{\min}$. $M_{\min} = 0.3$ is adopted [4]. This contingence corresponds to a three-phase short circuit on bus 39, with outage of the transmission line between busses 39 and 40.

It is emphasized that, the neural network can generate, transient stability restrictions (Eq. (8)) for a set of interest contingencies, e.g., for every contingency that presents security margin less than a predefined level. These restrictions can be aggregated to other restrictions of the system (power balance, physical limits of the electric equipment, etc.) and the preventive control problem solved by using some other optimization method.

For solving the preventive control, the generation reallocation and the load shedding is adopted. The neural network for the referred contingence produces a sensitivity coefficient of best performance (most absolute value) referred to the generation and load. For stabilizing the system, a load shedding on bus 40 and a reduction of generation on bus 9 is necessary. Table 3 shows the data related to carry out the security correction.

Suppose that the security margin of $\mathcal{M}_{12}^0 = -0.8188$ is desired to correct for $\mathcal{M}_i \geq M_{\min} = 0.3$. Then, the control action (generation reallocation/load shedding) gives a security margin displacement of $\Delta\mathcal{M} = 1.1188$. This goal is attained by sensibility analysis through a generation reallocation on bus 9 and a load shedding on bus 40, corresponding to 0.8 pu. The final security margin is 0.330, which is a good approximation of $\mathcal{M}_{12} \geq M_{\min}$.

Table 3
Analyzed results before and after the control action (generation reallocation and load shedding)

State	Item	Value
Initial	Initial security margin ($\mathcal{M}^0$)	−0.8188
Goal	Goal security margin ($M_{\min}$)	0.3
Security margin displacement	Necessary security margin variation ($\mathcal{M}^0 - M_{\min}$)	1.1188
Identification of the more sensible busses	More sensible generation bus	9
	More sensible load bus	40
Sensitivity analysis	Sensitivity coefficient of bus 9 ($\partial\mathcal{M}_r/\partial P_9$)	−1.1435
	Sensitivity coefficient of bus 40 ($\partial\mathcal{M}_r/\partial P_{40}$)	0.255
Control action	Generation reallocation at bus 9	−0.8
	Load shedding at bus 40	0.8
Final	Final security margin ($\mathcal{M}^f$)	0.330

Table 4
Transient stability study considering a generation and load profile corresponding to a pseudo-random distribution of 12.5% in relation to the base case, random number equal to 2520, active generation reduction on bus 9 and load shedding on bus 40 equal to 0.8 pu.

Contingency number	Data of the defect		Security margin ($\mathcal{M}$)
	Bus under short-circuit	Transmission Line removed	
1	17	16–17	0.8063
2	15	15–16	0.7169
3	36	25–36	0.8895
4	29	25–29	0.7679
5	15	14–15	0.7406
6	33	11–33	0.8304
7	29	29–30	0.8718
8	16	16–17	0.8148
9	16	16–18	0.7934
10	25	25–27	0.9413
11	30	29–30	0.9800
12	39	39–40	0.3301
13	13	13–45	0.4719
14	25	25–29	0.9342
15	25	25–26	0.9470

Table 4 shows the results after generation reallocation and load shedding.

Therefore, the generation reallocation produces a security margin correction of −0.8188 to 0.330, i.e., $\mathcal{M}_2 \geq M_{\min}$. The other contingencies also attend this requirement. The objectives are attained, i.e., $\mathcal{M}_i \geq M_{\min}$, $i = 1, 2, \ldots, 7$.

The neural network training is implemented using four different neural networks, according to Tables 5 and 6. They correspond to conventional back-propagation training, fuzzy back-propagation, conventional back-propagation training with adaptation of sigmoid function (translation and inclination) and

Table 5
Parameters of the neural networks used

	BP conventional	BP fuzzy
Number of pattern vectors	50	50
Number of layers	3	3
Number of neurons per layer	(43,55,1)	(43,55,1)
Sigmoid function	$\frac{1-\exp(-\lambda s_i)}{1+\exp(-\lambda s_i)}$	$\frac{1-\exp(-\lambda s_i)}{1+\exp(-\lambda s_i)}$
Training rate (γ^0)	0.07	0.07
Inclination of sigmoid function (λ^0)	0.3	0.3
Momentum term	No	No
Fuzzy controller inclusion	No	Yes
Exponential decline	No	0.75
Inclusion of the adaptation mechanism for λ and ρ	No	No
Training rate of the inclination of the sigmoid function	No	No
Training rate of the translation of the sigmoid function	No	No
Training time (s)	7379.70	6898.63
Execution time of the generation and analysis of the sensitivity model (s)	0.015	0.015

Table 6
Parameters of the neural networks used

	BP conventional with adaptation of inclination and translation of the sigmoid function	BP fuzzy with adaptation of inclination and translation of the sigmoid function
Number of pattern vectors	50	50
Number of layers	3	3
Number of neurons per layer	(43,55,1)	(43,55,1)
Sigmoid function	$\frac{1-\exp(-\lambda s_i)}{1+\exp(-\lambda s_i)}$	$\frac{1-\exp(-\lambda s_i)}{1+\exp(-\lambda s_i)}$
Training rate (γ^0)	0.07	0.07
Inclination of sigmoid function (λ^0)	0.3	0.3
Momentum term	No	No
Inclusion of the fuzzy controller	No	Yes
Exponential decline	No	0.65
Inclusion of the adaptation mechanism for λ and ρ	Yes	Yes
Training rate of inclination of sigmoid function	0.007	0.007
Training rate of translation of sigmoid function	0.1	0.1
Training time (s)	635.80	616.52
Execution time of generation and execution of the sensitivity analysis model (s)	0.015	0.015

fuzzy back-propagation with adaptation of sigmoid function (translation and inclination).

The four different neural networks give the same solution, and the differences are on the training time. Considering the back-propagation training, sometimes the convergence is very slow and could not converge, and the adaptations like fuzzy back-propagation or the inclination and translation of the sigmoid function always provide a solution with less time, and also less number of iterations during the training according to Tables 5 and 6.

9. Conclusion

A procedure for analyzing the transient stability and the preventive control of electric energy systems formulated using the sensitivity analysis generated by feedforward neural networks is proposed in this work. The neural network training is realized using the back-propagation algorithm with a fuzzy controller [13], associated with an adaptive process [15]. This algorithm gives a faster convergence and more accurate results [15], when compared to the traditional back-propagation algorithm, by adjusting the training rate and using the information of the error and the global error variation. The adaptation of the translation and inclination parameters also provides a guarantee of obtaining a solution by increasing the space of search and avoiding the paralysis process. Once the training is carried out, an off-line procedure, the neural network is able to estimate the security margin and the sensitivity analysis (a procedure effectuated almost on-line, and without computational effort). With these information, it is possible to develop a procedure for realizing the security correction (preventive control) for levels considered adequate for the system: in this work corresponds to $\mathcal{M} \geq M_{\min} = 0.3$.

References

[1] P.M. Frank, Introduction to System Sensitivity Theory, Academic Press, New York, 1978.
[2] A.A. Fouad, T. Jianzhong, Stability constrained optimal rescheduling of generation, IEEE Trans. Power Syst. 8 (1993) 105–112.
[3] P.M. Anderson, A.A. Fouad, Power system control and stability, second ed., IEEE Series on Power Engineering, USA, 2003.
[4] C.R. Minussi, W. Freitas Filho, Sensitivity analysis for transient stability, IEE Proc. Gener. Transm. Distribution 145 (1998) 669–674.
[5] V. Vittal, E.Z. Zhou, C. Hwang, A.A. Fouad, Derivation on stability limits using analytical sensitivity of the transient energy margin, IEEE Trans. Power Syst. 4 (1989) 1363–1372.
[6] Y. Xue, T. Van Cutsem, M. Ribbens-Pavella, Real-time analytic sensitivity method for fast transient stability assessment of large power systems, IEE Proc. 135 (1988) 107–116.
[7] S.V. Kartalopoulos, Understanding Neural Networks and Fuzzy Logic, IEEE Press, New York, 1996.
[8] P.K. Simpson, Artificial Neural Systems: Foundations, Paradigms, Applications and Implementations, Pergamon Press, New York, 1989.
[9] P.J. Werbos, Beyond regression: new tools for prediction and analysis in the behavioral sciences, Ph.D. thesis, Harvard University, 1974.
[10] B. Widrow, M.A. Lehr, 30 years of adaptive neural networks: perceptron, madaline, and backpropagation, Proc. IEEE 78 (1990) 1415–1442.
[11] P. Arabshahi, J.J. Choi, R.J. Marks II, T.P. Caudell, Fuzzy parameter adaptation in optimization, IEEE Comput. Sci. Eng. (Spring) (1996) 57–65.
[12] T. Athay, V.R. Sherkat, R. Podmore, S. Virmani, C. Pench, Transient energy stability analysis, in: Conference on System Engineering for Power: Emergency Operating State Control—Section IV, U.S. Department of Energy Publication, Davos, Switzerland, 1979 (No. CONF-79004-P1).
[13] M.L.M. Lopes, C.R. Minussi, A.D.P. Lotufo, Electric load forecasting formulation by a fast neural network, Eng. Intell. Syst. 11 (2003) 51–57.
[14] T. Terano, K. Asai, M. Sugeno, Fuzzy Systems Theory and its Application, Academic Press, NY, USA, 1987.
[15] M.L.M. Lopes, A.D.P. Lotufo, C.R. Minussi, A fast electric load forecasting using adaptive neural networks, Bologna, Itália, IEEE Bologna Power-Tech 2003, 2003.
[16] N. Stamatis, D. Parthimos, T.M. Griffith, Forecasting chaotic cardiovascular times series with an adaptive slope multilayer perceptron neural network, IEEE Trans. Biomed. Eng. 46 (1999) 1441–1453.
[17] L.G. Fonseca, I.C. Decker, Iterative algorithm for critical energy determination in transient stability of power system, in: IFAC—Symposium Planning and Operation in Electric Energy System, RJ, Brazil, 1985, pp. 483–489.
[18] M. Kezunovic, A. Abur, A. Edris, D. Sobajic, Data integration/exchange, IEEE Power Energy 2 (2004) 14–19.
[19] A.D.P. Lotufo, Sensitivity analysis by neural networks for transient stability studies of electrical power systems, Ph.D. thesis, UNESP, Ilha Solteira, SP, 2004 (in portuguese).
[20] D. Novosel, M.M. Begovic, V. Madani, Shedding light on blackouts, IEEE Power Energy 2 (2004) 32–43.
[21] M.A. Pai, Power System Stability, North Holland Control Series, 1981.
[22] C.R. Minussi, M.C.G. Silveira, Transient stability analysis of electric power systems by neural networks, in: 38rd Midwest Symposium on Circuits And Systems, Rio de Janeiro, RJ, 1995, pp. 1305–1308.

Anna Diva P. Lotufo graduated in electrical engineering from UFSM, Santa Maria, RS, Brazil in 1978, M.Sc. from UFSC, Florianópolis, SC, Brazil, in 1982 and Ph.D. from UNESP, Ilha Solteira, SP, in 2004. She is currently an assistant professor at UNESP, Ilha Solteira, SP, Brazil, doing research in transient and preventive control of electrical power system area using neural networks. E-mail: annadiva@dee.feis.unesp.br.

Mara Lúcia M. Lopes graduated in mathematics from UFMS, Três Lagoas, MS, Brazil, in 1997. Received her M.Sc. degree from UNESP, Ilha Solteira, SP, Brazil in 2000. She is presently a Ph.D. student at UNESP, Ilha Solteira, SP, Brazil, doing research in load forecasting by neural network area. E-mail: mara@dee.feis.unesp.br.

Carlos R. Minussi graduated in electrical engineering from UFSM, Santa Maria, RS, Brazil in 1978, M.Sc. and Ph.D. from UFSC, Florianópolis, SC, Brazil, in 1981 and 1990, respectively. He is currently an associate professor at UNESP, Ilha Solteira, SP, Brazil. His main interests are in analysis, control of power systems and neural networks. E-mail: minussi@dee.feis.unesp.br.

Available online at www.sciencedirect.com
SCIENCE DIRECT®
Applied Soft Computing 5 (2005) 235–244
Applied Soft Computing
www.elsevier.com/locate/asoc

Electric load forecasting using a fuzzy ART&ARTMAP neural network

Mara Lúcia M. Lopes*, Carlos R. Minussi, Anna Diva P. Lotufo

Department of Electrical Engineering, São Paulo State University (UNESP), P.O. Box 31, Zip 15385-000, Ilha Solteira, SP, Brazil

Received 13 January 2004; received in revised form 15 July 2004; accepted 20 July 2004

Abstract

This work presents a neural network based on the ART architecture (adaptive resonance theory), named fuzzy ART&ARTMAP neural network, applied to the electric load-forecasting problem. The neural networks based on the ART architecture have two fundamental characteristics that are extremely important for the network performance (stability and plasticity), which allow the implementation of continuous training. The fuzzy ART&ARTMAP neural network aims to reduce the imprecision of the forecasting results by a mechanism that separate the analog and binary data, processing them separately. Therefore, this represents a reduction on the processing time and improved quality of the results, when compared to the Back-Propagation neural network, and better to the classical forecasting techniques (ARIMA of Box and Jenkins methods). Finished the training, the fuzzy ART&ARTMAP neural network is capable to forecast electrical loads 24 h in advance. To validate the methodology, data from a Brazilian electric company is used.
© 2004 Elsevier B.V. All rights reserved.

Keywords: Adaptive resonance theory; Electric load forecasting; Electric power systems; Neural networks; Fuzzy logic; Fuzzy ART&ARTMAP neural network

1. Introduction

Expansion planning, load flow, economic operation, security analysis and electric energy systems control are some of studies that effectively depend on the behaviour of the load profile [1], i.e., forecasting of future information of a time series based on passed values. In the literature, there are many methods to effectuate load forecasting, for example, multiple or simple linear regression, exponential smoothing, state estimation, Kalman filter, ARIMA of Box&Jenkins methods [1–3]. All of these methods need a previous modelling to be applied a posteriori. It is necessary to know some information as for example: cloudy days, wind velocity, sudden temperature variation and non-conventional day effects (holidays, strikes, etc.), to model the load.

* Corresponding author. Tel.: +55 18 37431222; fax: +55 18 37431163.
E-mail addresses: mara@dee.feis.unesp.br (M.L.M. Lopes), minussi@dee.feis.unesp.br (C.R. Minussi), annadiva@dee.feis.unesp.br (A.D.P. Lotufo).

1568-4946/$ – see front matter © 2004 Elsevier B.V. All rights reserved.
doi:10.1016/j.asoc.2004.07.003

After modelling the load, and using this information, the algorithm is initialised to take the results.

The use of neural networks is nowadays an alternative method that is becoming an efficient technique to solve the electric load-forecasting problem.

There are researches that treat this problem using the Back-Propagation algorithm [4,5]. This algorithm is considered on the specialised literature a benchmark in precision. However, the convergence is slow, although there are some adaptations to improve the performance [4,6]. Therefore, the idea is to use a neural network that combines good results with a faster processing. The ART neural networks are architectures based on adaptive resonance theory, having two fundamental characteristics for the network processing: the stability and the plasticity (capacity of learning new patterns without loosing the memory or the patterns already trained) [7]. These two characteristics provide to the network a stable learning and a faster convergence [8]. The objective of this work is to develop another neural network based on the ART architecture, called fuzzy ART&ARTMAP neural network [9], and apply it to the electric load-forecasting problem [10–12].

The fuzzy ART&ARTMAP neural network is composed of two modules: (1) fuzzy ART and (2) fuzzy ARTMAP. The ART network is architecture based on a supervised training for multi dimensional mapping (several inputs and several outputs), composed of two ART modules connected to an inter-ART module [7]. The fuzzy ART module classifies the input vectors in categories, composed of analog data that are converted to binary values by an active/binary code converter. The input set of fuzzy ARTMAP module is composed of binary data (that have already been converted in binary), plus the binary information referred to time (month, day of the week, holiday, and hour, etc). The fuzzy ARTMAP output module is composed of electric load values referred to the subsequent hour, and are processed in classes/areas and codified in binary. This way, this network aims to solve, or at least reduce the imprecision of the forecasting results, by a mechanism that separate the analog and binary data, and processing them separately. The fuzzy ARTMAP module receives only binary data, allowing applications in large-scale systems. It is important to emphasise that this architecture preserves the plasticity, allowing the implementation of continuous training, thus becoming an important tool for electric load forecasting.

To illustrate the proposed method, an example is presented considering historical data from a Brazilian electric company.

It is emphasised that, in this work, for convenience the vectors are represented by rows and not by columns (as usually).

2. ART (Adaptive Resonance Theory)

In 1976, Grossberg introduced the adaptive resonance theory [13,14], concept as a cognitive human information process, through a self-organisation, effectuating stable category recognition and answering an arbitrary sequence of input patterns [7,15–17]. This kind of network has a self-organisation and a self-stabilisation that allows solving the stability–plasticity dilemma. Thus, the ART network is capable to assimilate new things while maintaining those already instructed [15].

The ART network structure is composed of three layers as shown on Fig. 1 [18]:

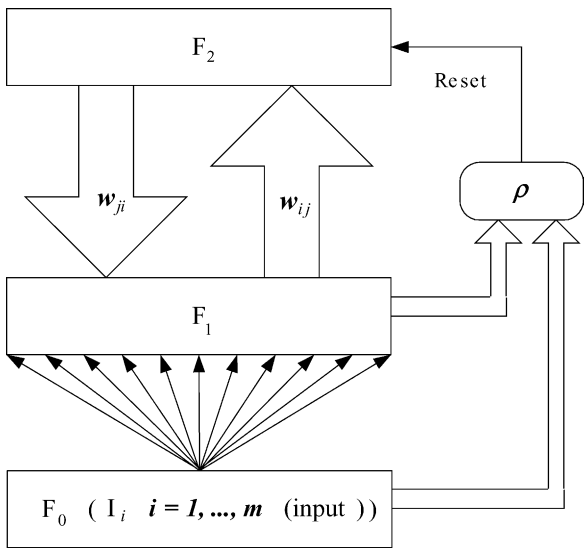


Fig. 1. ART network structure.

- F_0: input layer (receives and stores the new input patterns);
- F_1: comparing layer (intensification of the difference and elimination of the noises of the input patterns);
- F_2: recognition layer (store the categories (clusters)).

There are two essential memories to execute the ART neural network, according to [14]. They are:

1. LTM (long term memory): weight connection between the layers:
   $F_1 \rightarrow F_2$ (bottom-up);
   $F_2 \rightarrow F_1$ (top-down);
2. STM (short term memory): activity patterns developed on the layers F_1 and F_2.

The performance of the ART network is effectuated as follows [19]:

1. An input pattern is presented to the network;
2. If the input pattern is similar to patterns already stored on the ART network LTM $\rightarrow$ the input pattern pertains to a category already formed;
3. If the input pattern is not similar to any pattern already stored on the ART network LTM $\rightarrow$ the input pattern does not pertain to any category already formed, and the network will form a new category, which will store the new input pattern.

The vigilance parameter is responsible for identifying this similarity [19].

2.1. Fuzzy ART

This architecture has a non-supervised training and uses structure calculus based on fuzzy logic.

The fuzzy ART network can be considered as a modification of ART1 network. Input patterns of fuzzy ART are binary data as well as analog data, but the ART1 network uses only binary data.

2.2. Fuzzy ART algorithm

The principal steps of the fuzzy ART algorithm are resumed on the following procedures.

2.2.1. Input data

The input pattern vector of the F_0 layer is represented by the vector $\mathbf{a}$, $\mathbf{a} = [\mathbf{a}_1 \ \ldots \ \mathbf{a}_M]$, M-dimensional. This vector is normalised to avoid category proliferation, according to the equation:

$$\bar{\boldsymbol{a}} = \frac{\boldsymbol{a}}{|\boldsymbol{a}|} \tag{1}$$

where:

$$|\boldsymbol{a}| \equiv \sum_i a_i$$

and $\bar{\boldsymbol{a}}$ normalised input vector.

Afterwards, it is effectuated the codification of the complement, preserving the amplitude information. The complement codification is represented as Eq. (2) [18]:

$$\bar{a}_i^c = 1 - \bar{a}_i \tag{2}$$

where $\bar{a}^c$ is complement of the normalised vector.

Then, the input vector of the F_0 layer, is a $2M$-dimensional vector denoted by Eq. (3) [11,18]:

$$\boldsymbol{I} = [\bar{\boldsymbol{a}}\, \bar{\boldsymbol{a}}^c] \equiv [\bar{a}_1 \cdots \bar{a}_M \cdots \bar{a}_1^c \cdots \bar{a}_M^c] \tag{3}$$

The activity vector of F_1 is indicated by $\mathbf{x} = [x_1 \ldots x_M]$, according to Eq. (4) described below:

$$x = \begin{cases} I & \text{if } F_2 \text{ is inactive} \\ I \wedge w_J & \text{if the } J\text{th node of } F_2 \text{ is active} \end{cases} \tag{4}$$

The activity vector of F_2 is represented by $\mathbf{y} = [y_1 \ \ldots \ y_N]$, where N is the number of the categories created in F_2. Thus, according to [20]:

$$\begin{aligned} y_J &= 1 \\ y_j &= 0 \quad \text{for } j \neq J. \end{aligned} \tag{5}$$

2.2.2. Parameters

There are three fundamental parameters for the performance and learning of the fuzzy ART network according to [14,21]:

1. Chosen parameter ($\alpha > 0$): acts on the category selection;
2. Training rate ($\beta \in [0,1]$): controls the velocity of the network adaptation;
3. Vigilance parameter ($\rho \in [0,1]$): controls the network resonance. The vigilance parameter is the responsible of the number of the categories formed

in F_2. If ρ is very large, it produces a good classification, providing the generation of several categories (the network has less errors but has less capacity of generalisation). If ρ is very small, it will generate few categories (the network will have more capacity of generalisation, but more possibility of making mistakes).

2.2.3. Weight initialisation

Initially every weight has the same value equal to 1, i.e.:

$$w_{j1}(0) = \cdots = w_{jM}(0) = 1 \tag{6}$$

showing that there is no active category.

2.2.4. Choice category

Given the input vector **I** in F_1, for each node j in F_2, i.e., for each category, the choice function T_j is determined by Eq. (7) [18,19]:

$$T_j = \frac{|\boldsymbol{I} \wedge w_j|}{\alpha + |w_j|} \tag{7}$$

where, $\wedge$ is the fuzzy operator AND, defined by Eq. (8):

$$(\boldsymbol{p} \wedge \boldsymbol{q}) \equiv \min\ (p_i, q_i) \tag{8}$$

The choice is realised when at least one of the nodes in F_2, become active. The category is chosen as the active node J according to [18,21]:

$$J = \underset{j \in 1,\ldots,N}{\arg\max}\ T_j \tag{9}$$

If there is more than one active category, the chosen category is that one with the smaller index.

2.2.5. Resonance or reset

Resonance occurs if the vigilance criterion Eq. (10) is satisfied:

$$\frac{|\boldsymbol{I} \wedge w_J|}{|\boldsymbol{I}|} \geq \rho \tag{10}$$

If Eq. (10) is not satisfied, then the reset occurs. On the reset, the node J of F_2 is excluded from the search process of the active category given by Eq. (9). This is, T_J assumes a value equal to zero and it is chosen a new category by Eq. (9) to realise the resonance process. This procedure is executed until the network finds a category that satisfies Eq. (10).

2.2.6. Learning

After the input vector **I** has completed the resonance state, it follows the learning process, where occurs the modification of the weight vector given by Eq. (11) [19,20]:

$$w_J^{\text{new}} = \beta(I \wedge w_J^{\text{old}}) + (1 - \beta) w_J^{\text{old}} \tag{11}$$

where J is active category.

Two types of training dependent of the value of β, are possible:

1. Fast training: $\beta = 1$, permits the system to adapt faster;
2. Slow training: $0 < \beta < 1$, obliges the weights of the system to adapt slowly.

2.3. Fuzzy ARTMAP

The fuzzy ARTMAP architecture is an auto-organised learning system [22,23]. This kind of network has supervised training and pertains to the ART family, i.e., the structure is based on the adaptive resonance theory and is similar to the fuzzy ART network, employing calculus based on the fuzzy logic.

This network is composed of two fuzzy ART modules: ART_a and ART_b, interconnected by an inter-ART by an associative memory module as illustrated on Fig. 2 [7]. The inter-ART module has a self-regulator mechanism named match tracking, whose objective is to maximise the generalisation and minimise the network error [24].

The F_2^a layer is connected to the inter-ART module by the weights w_{jk}^{ab}, assuming initially a value equal to one ($w_{jk}^{ab} = 1$), showing that there is no association with the ART_a and ART_b modules [20].

2.4. Fuzzy ARTMAP algorithm

The steps of the fuzzy ARTMAP algorithm are summarised below.

2.4.1. Input data

The input pattern vector of the ART_a network is represented by the vector $\boldsymbol{a} = [a_1 \ldots a_{M_a}]$, M_a-dimensional, and the input pattern vector of the ART_b network (corresponding to the desired output referred

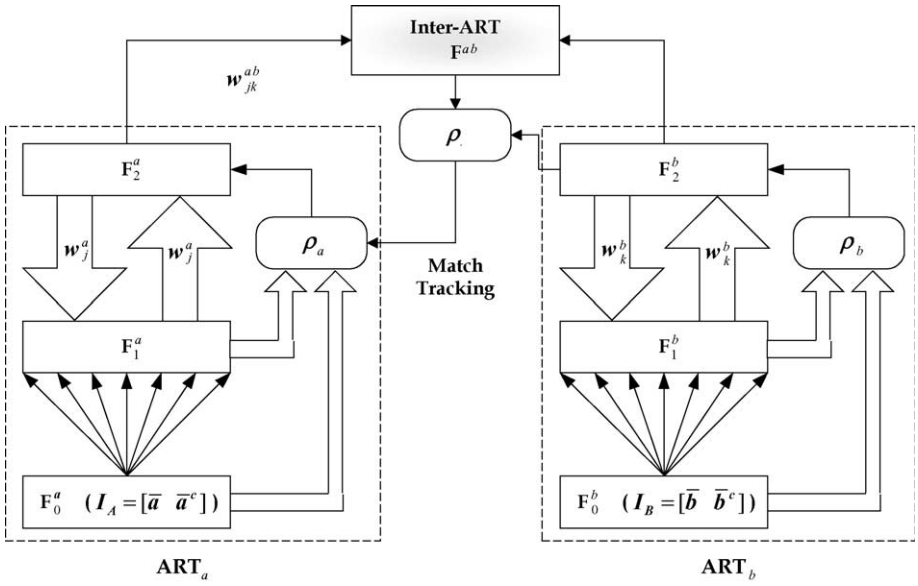


Fig. 2. Structure of the ARTMAP network.

to the input pattern of ART_a network) is represented by vector $\boldsymbol{b} = [b_1 \ldots b_M]$, M_b-dimensional [22].

2.4.2. Parameters

The parameters used on the fuzzy ARTMAP network are the same as those of the fuzzy ART network, and receive a pattern for each specific module. The difference of this network is correspondent to the parameters ρ_{ab}, $\rho_{ab} \in [0,1]$, representing the vigilance parameter referred to the inter-ART module.

2.4.3. Algorithm structure

The ARTMAP network effectuates the processing of two ART networks (ART_a and ART_b), and after the resonance is confirmed in each network, one has:

- J, active category for the ART_a network;
- K, active category for the ART_b network.

The next step is to verify using the match tracking, if the active category on ART_a corresponds to the desired output vector presented on ART_b. The vigilance criterion is given by Eq. (12):

$$\frac{\left|y^{\mathrm{b}} \wedge w^{\mathrm{ab}}_{\mathrm{Jk}}\right|}{\left|y^{\mathrm{b}}\right|} \geq \rho_{ab} \tag{12}$$

where y^{b} is ART_b output vector (activity pattern in F^b_2).

If Eq. (12) is not satisfied, it is effectuated a little increment on the vigilance parameter ART_a, only enough to exclude that category and select another category which will be active and reintroduced on the process until Eq. (12) be satisfied.

2.4.4. Learning

After occurring the resonance by the vigilance criterion Eq. (12), it is effectuated the weight adaptation. The adaptation of the ART_a and ART_b module weights, is the same as that of fuzzy ART network, and the adaptation for the inter-ART module is effectuated as follows [20]:

$$w^{\mathrm{ab}}_{\mathrm{JK}} = 1$$

$$w^{\mathrm{ab}}_{\mathrm{Jk}} = 0 \quad \text{for } k \neq K$$

3. Fuzzy ART&ARTMAP neural network

Neural networks based on the resonance adaptive theory have architecture characteristics, which are extremely important for systems that need immediate answers. Therefore, this is the great interest of using this kind of network, exploring the characteristics of

stability and plasticity, particularities of the ART family networks. One of the few disadvantages of the ART networks can be the great sensibility in relation to the parameters and the complexity of analysing the results obtained by the network.

This two kind of networks has a great flexibility, which allows idealising new architectures (applied to specific problems). The architectures are based on the adaptive resonance theory to solve the problems found on the ART architecture in relation to the category chosen criterion and the vigilance test, which cause imprecision on the ART network.

This way, the objective of this work is to develop an architecture based on the adaptive resonance theory, being capable to solve or at least to reduce the imprecision of load forecasting results, by a mechanism that separate the analog and binary data, and processing separately.

The proposed network is called fuzzy ART&ARTMAP and is composed of two modules:

1. Fuzzy ART;
2. Fuzzy ARTMAP.

The function of ART module is to classify the input vectors in categories composed of analog data (electrical load values), which are converted in binary values by an analog/binary converter (A/B). The ARTMAP module input data is composed of these binary data plus a set of other binary information referred to: month, day of the week, holiday, hour, etc.

The output of ARTMAP module is constituted of electrical load values referred to the subsequent hour. These load values are normalised by a maximum pattern load value, pertaining to the interval between 0 and 1. This interval [0,1] is partitioned in 127 classes, and each class is divided in 15 areas. The normalised load values are identified inside the classes and the areas, therefore a respective area and class represent each load value. Then, the values are converted in binary by an A/B converter and used as an output set of the ARTMAP module.

The ARTMAP module receives only binary data, allowing applications in large systems. This, represents a reduction on the processing time and a quality of the results, when compared to the Back-Propagation neural network and better to the classical forecasting methods (ARIMA of Box&Jenkins, and others) [1].

It is emphasised that this conception preserves the characteristic of plasticity, therefore allowing the implementation of a continuous training that consists of a very important tool for electrical power system operation, in special for electrical load forecasting. Fig. 3 illustrates the fuzzy ART&ARTMAP neural network.

4. Electrical load forecasting

Short term electrical load forecasting (daily forecasting) is executed as follows:

- The output of a determined instant is used as input for the subsequent instant.
- The network input pattern data is divided in two parts:
  1. The first part is the input pattern of the ART module and is composed of historic data (hour) predefined in an interval, for example in a month. In a determined hour h, the input is defined as the load values extracted from the historic data (current hour, 1, 2 and 3 hours before the current one).
  2. The second part corresponds to the input pattern data, which with the binary data form the ART module from the input set of the ARTMAP module. These data are referred to the time (month, day of the week, holiday or not, hour, etc.), all them in a binary form. The results can be improved including other variables like: temperature, humidity, wind velocity, cloudy day, etc., nevertheless it is not used in this work.

The network output corresponds to the load value referred to the hour $(h + 1)$, which is processed in classes and areas and afterwards codified in binary code.

The input patterns of the network are represented as follows:

$$P^{\text{an}}(h) = [L(h-p)\cdots L(h-2)L(h-1)L(h)], \quad P^{\text{an}} \in R^m$$

$$\Lambda^{\text{bin}}(h) = [t], \quad \Lambda^{\text{bin}} \in R^n$$

where $\mathbf{P}^{\text{an}}(h)$ is analog input data; $\boldsymbol{\Lambda}^{\text{bin}}(h)$, binary input data; m, dimension of the $\mathbf{P}^{\text{an}}$ vector; n, dimension of

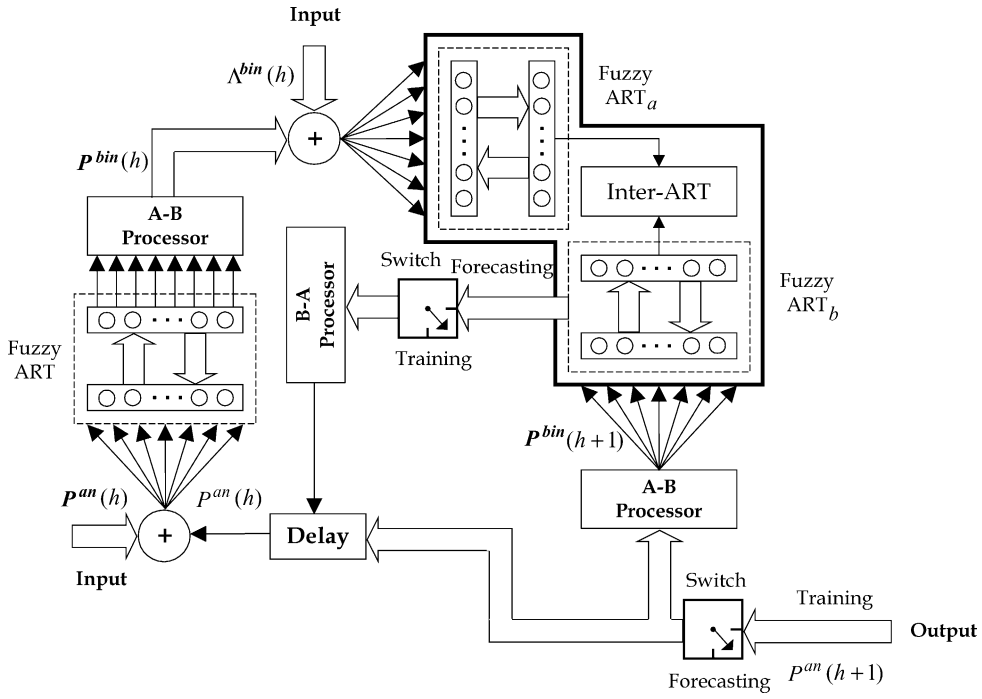


Fig. 3. Fuzzy ART&ARTMAP network.

the $\boldsymbol{\Lambda}^{\text{bin}}$ vector; $L(h-p)$, load value, p hours before the current hour h; $\mathbf{t}$, time vector referred to the historic data (month, day of the week, holiday, hour, etc.).

The vector $\mathbf{P}^{\text{an}}(h)$ gets in the fuzzy ART module, which classifies in categories. In the sequence, these categories are converted in binary providing the vector $\mathbf{P}^{\text{bin}}(h)$.

The load values representing the network output pattern are normalised according to Eq. (13), where $L_n(h+1)$ pertains to the interval [0,1].

$$L_n(h+1) = \frac{L(h+1)}{\text{CMAX}} \tag{13}$$

where, $L_n(h+1)$, normalised electrical load referred to the hour subsequent to the current hour h; $L(h+1)$, electrical load value corresponding to the hour subsequent to the current hour h; CMAX, maximum load value permitted.

The interval [0,1] is discretized in 127 classes, and each class is divided in 15 areas. This avoids the network output to provide load values alike. Then, the network can identify the load value as close as possible to the reality. This way, one class, and one area represent each $L_n(h+1)$, respectively:

$$\boldsymbol{L}_n^{\text{bin}}(h+1) = [\underbrace{101\ldots0}_{\boldsymbol{C}^{\text{bin}}}\underbrace{001\ldots1}_{\boldsymbol{A}^{\text{bin}}}]$$

where is $\mathbf{C}^{\text{bin}}$, vector of classes representing $L_n(h+1)$; $\mathbf{A}^{\text{bin}}$, vector of areas representing $L_n(h+1)$; $\boldsymbol{L}_n^{\text{bin}}(h+1)$, binary representation of $L_n(h+1)$.

Each class and area is converted in binary code. This way, the input and output vectors that compose the fuzzy ART&ARTMAP are illustrated on the following way:

$$\boldsymbol{X}(h) = [\boldsymbol{P}^{\text{bin}}(h)\Lambda^{\text{bin}}(h)], \quad \boldsymbol{Y} \in R^{\text{mb}+n}$$

$$\boldsymbol{Y}(h) = L_n^{\text{bin}}(h+1), \quad \boldsymbol{Y} \in R^{\text{cb}+\text{ab}}$$

where mb is dimension of vector $\mathbf{P}^{\text{bin}}$; cb, dimension of vector $\mathbf{C}^{\text{bin}}$; ab, dimension of vector $\mathbf{A}^{\text{bin}}$.

Electrical loads $L(h-p), \ldots, L(h-1)$ represent the feedback loop with a delay on the output. The data used on the training referred to the time (for each vector) are day of the week and hour of the day.

Table 1
Specification of the fuzzy ART&ARTMAP neural network

Item	Value
Number of pattern vectors	504
Training rate, β	1.00
Choice parameter, α	1.30
Vigilance parameter (ART_a), ρ_a	0.94
Vigilance parameter (ART_b), ρ_b	0.99
Vigilance parameter (inter-ART), ρ_{ab}	1.00

Considering the input vector representation which pertains to the fuzzy ART&ARTMAP network such that: $\mathbf{X}(h)$, dimension 20, $[\mathbf{P}^{\text{bin}}(h)\ \boldsymbol{\Lambda}^{\text{bin}}(h)]$, with $\mathbf{P}^{\text{bin}}(h)$ with dimension 12 and $\boldsymbol{\Lambda}^{\text{bin}}(h)$ dimension 8, and $\mathbf{Y}(h)$ dimension equal to 11.

The historical data are the electrical load and were obtained from a Brazilian electric company. These data contain hour load from the year 1998, and it is considered the non-typical days (holidays) and special days (Saturday and Sundays) and days from a typical week. The historic data used to effectuate the training of the fuzzy ART&ARTMAP network are between 8 and 28 July, 1998.

Therefore, a vector with 504 inputs and respective outputs represents the data set. Table 1 shows the principal parameters used on the training and diagnosis of the developed neural network.

Fig. 4 shows the load forecasting results (real data and fuzzy ART&ARTMAP neural network) referred to 29 July, 1998. The mean absolute percentile error

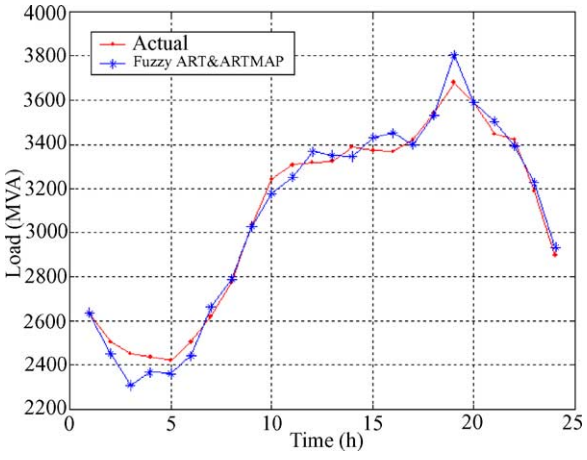


Fig. 4. Load forecasting results.

Table 2
Comparative results

Item	Conventional Back-Propagation	Fuzzy ART&ARTMAP
Processing time (s)	1165.98	2.04
Gain GT	–	571.56
MAPE (%)	1.80	1.62
MAX (%)	6.20	5.68

(MAPE) (14) [25] and the maximum error (MAX) Eq. (15) are defined to improve the analysis of the results for the daily forecasting. This way, they are shown below:

$$\text{MAPE} = \frac{1}{N_h}\left\{\sum_{i=1}^{N}\frac{(|L(h) - \underline{L}(h)|)}{L(h)}\right\} \times 100 \tag{14}$$

$$\text{MAX}(\%) = \max\left\{\frac{|L(h) - \underline{L}(h)|}{L(h)}\right\} \times 100 \tag{15}$$

where $L(h)$ is real value of the load referred to the hour h; $\underline{L}(h)$, estimated value of the load referred to the hour h; N_h, total number of hours.

The execution time gain (processing) Eq. (16) is defined to analyse the performance of the developed methodology, considering the necessary time to effectuate the training and the diagnosis, on the following way:

$$\text{GT} = \frac{\text{TBP}}{\text{TFA}} \tag{16}$$

where TBP is Back-Propagation conventional execution time (s); TFA, fuzzy ART&ARTMAP execution time (s).

The program was elaborated in FORTRAN and processed in a Pentium 4 (1.7 GHz and 256 MB of RAM memory). The processing time is referred only to the execution of the algorithm, excluding the operations of data reading and data output. Comparative results are shown on Table 2.

5. Conclusion

It is developed a procedure for electrical load forecasting by a fuzzy ART&ARTMAP neural network. The network is an innovation among the ART

architecture existent on the literature. In this network all the processing is based on binary data, which facilitate the application in large systems and permitting the network processing on-line. The presented results use data from a Brazilian electrical company. The load forecasting is executed considering one day ahead. The ART&ARTMAP network is advantageous face to the Back-Propagation network considering the processing time [4,6]. The mean absolute percentile error (MAPE) and the maximum daily forecasting error are 1.62 and 5.68%, respectively. Concluding, the formulation presented in this work provide the following advantages:

- Reduction on the processing time;
- The plasticity characteristic is preserved;
- Application in large scale systems;
- Implementation in continuous systems;
- Results with better quality.

Acknowledgments

The authors would like to acknowledge the financial support of FAPESP (Fundação de Amparo à Pesquisa do Estado de São Paulo)—Brazil (Proc. No. 00/15120-1).

References

[1] T.M. O'Donovan, Short Term Forecasting: An Introduction to the Box-Jenkins Approach, John Wiley & Sons, New York, 1983.
[2] C. Almeida, P.A. Fishwich, Z. Tang, Time Series Forecasting Using Neural Network vs. Box-Jenkins Methodology, Simulation Councies Inc., 1991.
[3] A. Abraham, B. Nath, A neuro-fuzzy approach for modelling electricity demand in Victoria, Appl. Soft Comput. 1 (2001) 127–138.
[4] M.L.M. Lopes, C.R. Minussi, A.D.P. Lotufo, Electrical load forecasting formulation by a fast neural network, Eng. Intell. Syst. 11 (1) (2003) 51–57.
[5] P.J. Werbos, Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences, Master Thesis, Harvard University, 1974.
[6] M.L.M. Lopes, A.D.P. Lotufo, C.R. Minussi, A fast electric load forecasting using adaptive neural networks, in: Proceedings of the 2003 IEEE Bologna Power-Tech, Italy, 2003.
[7] G.A. Carpenter, S. Grossberg, A self-organizing neural network for supervised learning, recognition and prediction, IEEE Commun. Magazine (1992) 38–49.
[8] E.P. Hernández, E.G. Sánchez, Y.A. Dimitriadis, Study of distributed learning as a solution to category proliferation in fuzzy ARTMAP based neural system, Neural Networks 16 (7) (2003) 1039–1057.
[9] W.P. Ferreira, Contingency Dynamic Analysis of Electric Power System by Neural Network Based on Adaptive Resonance Theory, Ph.D. Thesis, Department of Electrical Engineering, University of São Paulo State, Ilha Solteira, Brazil, 2003.
[10] V. Petridis, A. Kehagias, Predictive Modular Neural Network: Applications to Times Series, Klüwer Academic Publishers, Boston, 1998.
[11] S.E. Skarman, M. Georgiopoulos, A.J. Gonzalez, Short-term electrical load forecasting using a fuzzy ARTMAP neural network, in: Proceedings of the SPIE, Conference on Applications and Science of Computational Intelligence, Orlando, 1998, pp. 181–191.
[12] O.A.S. Carpinteiro, A.J.R. Reis, A.P.A. da Silva, A hierarchical neural model in short-term load forecasting, Appl. Soft Comput. 4 (2004) 405–412.
[13] S. Grossberg, Adaptive pattern classification and universal recoding, I: parallel development and coding of neural feature detectors, Biol. Cybernetics 23 (1976) 121–134.
[14] S. Grossberg, adaptive pattern classification and universal recoding, II: feedback, expectation, olfaction, and illusions, Biol. Cybernetics 23 (1976) 197–202.
[15] R.A. Baxter, Error propagation and supervised learning in adaptive resonance networks, in: Proceedings of the 1991 International Joint Conference on Neural Networks, 1991, pp. 423–429.
[16] G.A. Carpenter, S. Grossberg, Adaptive resonance theory, in: M.A. Arbib (Ed.), The Handbook of Brain Theory and Neural Networks, second ed., MA: MIT Press, Cambridge, 2003, pp. 87–90.
[17] G.A. Carpenter, Default ARTMAP, in: Proceedings of 2003 International Joint Conference on Neural Networks, 2003, pp. 1396–1401.
[18] J. He, A-H. Tan, C-L. Tan, ART-C: a neural architecture for self-organization under constraints, in: Proceedings of the 2002 International Joint Conference on Neural Networks, 2002, pp. 2550–2555.
[19] M.V. Butz, S. Ray, Bidirectional ARTMAP: an artificial mirror neuron system, in: Proceedings of the 2003 International Joint Conference on Neural Networks, 2003, pp. 1417–1422.
[20] S.I. Verzi, G.L. Heileman, M. Georgiopoulos, G.C. Anagnostopoulos, in: Proceedings of the 2003 International Joint Conference on Neural Networks, Universal approximations with fuzzy ART and fuzzy ARTMAP 3 (2003) 1987–1992.
[21] G.A. Carpenter, S. Grossberg, D.B. Rosen, Fuzzy ART: fast stable learning and categorization of analog patterns by an adaptive resonance theory, Neural Networks 4 (1991) 759–771.
[22] G.A. Carpenter, S. Grossberg, N. Markuzon, J.H. Reynolds, D.B. Rosen, Fuzzy ARTMAP: a neural network architecture for incremental supervised learning of analog multidimensional maps, IEEE Transac. Neural Networks 3 (5) (1992) 698–713.
[23] R. Palaniappan, R. Paramesran, Using genetic algorithm to identify the discriminatory subset of multi-channel

spectral bands for visual response, Appl. Soft Comput. 2 (2002) 48–60.

[24] C.P. Lim, R.F. Harrison, Modified fuzzy ARTMAP approaches bayes optimal classification rates: an empirical demonstration, Neural Networks 10 (4) (1997) 755–774.

[25] D. Srinivasan, S.S. Tan, C.S. Chang, E.K. Chan, Practical implementation of a hybrid fuzzy neural network for one-day-ahead load forecasting, IEE Proc. Gen. Trans. Distrib. 145 (6) (1998) 687–692.

Eng Int Syst (2003) 1: 51–57
© 2003 CRL Publishing Ltd

Engineering
Intelligent Systems

Electrical load forecasting formulation by a fast neural network

Mara Lúcia M Lopes, Carlos R Minussi and Anna Diva P Lotufo

Departamento de Engenharia Elétrica, Universidade Estadual Paulista, UNESP, Ilha Solteira, SP, Brazil
Email: mara@dee.feis.unesp.br

The objective of this work is to develop a methodology for electric load forecasting based on a neural network. Here, backpropagation algorithm is used with an adaptive process that based on fuzzy logic and using a decaying exponential function to avoid instability in the convergence process. This methodology results in fast training, when compared to the conventional formulation of backpropagation algorithm. The results are presented using data from a Brazilian Electric Company, and shows a very good performance for the proposal objective.

Keywords: Load forecasting, short term, neural networks, backpropagation, fuzzy logic

1. INTRODUCTION

Expansion planning, load-flow, economic operation and unit commitment, security analysis and control of electric energy systems are effectively dependent on the previous behavior of load profile (O'Donovan [5]), i.e. forecasting of future information of a time series based on past values. Recently with the arrival of competitive power markets, load forecasting importance has strongly been considered (Bunn [3], Doulai [6], Wang [19]). In technical literature many methods are implemented for short term load forecasting: simple or multiple linear regression, exponential smoothing, state estimation, Kalman filter, ARIMA of Box and Jenkins (Almeida *et al.* [1]). All these methods, need a previous load modeling for further application. To model the load it is necessary to know some information like: cloudy days, wind speed, temperature variations and effects of non-conventional days (holidays, strikes, etc).

Having obtained the load using these informations, an appropriated algorithm is used to get the results. The use of neural networks nowadays is a very efficient method mainly to the special days case, (Hippert *et al.*, [7]). In the specialized literature there are available several references that deal with load forecasting problems by neural networks (Charytoniuk and Chen [4], Doulai and Cahill [6], Hippert *et al.* [7], Kodogianis and Anagnostakis [11] and [12], Liang and Cheng [13], Osowski and Siwek [15], Srinivasan *et al.* [16], Tamimi and Egbert [17], Wang at al. [19], among others). Nevertheless, the objective of this work is to develop a methodology to electric load forecasting using an artificial neural network (ANN) (Kartalopoulos [10]) with training by backpropagation (BP) (Werbos [20]). The BP algorithm is considered, in technical literature, as a benchmark in precision. However the convergence using BP algorithm is quite slowly. The objective proposal of this work is to adjust the training rate γ during the convergence process, to reduce the execution time. The γ adjustment is done using a fuzzy controller. We also use a decaying exponential function that establishes a priority in the regulator actuation in the initial training time and avoid instability in the convergence process.

It must be emphasized that, in this work, we intend to

develop a methodology to short term electric load forecasting based on feedforward neural networks where the training is done by backpropagation algorithm (backpropagation neural network), that is efficient in precision and speed, when compared with the conventional backpropagation training. The comparison with backpropagation neural networks and traditional prediction methods (Box-Jenkins, etc.) have been done in (Almeida *et al.* [1], Charytoniuk and Chen [3], Tamimi and Egbert [17]), where neural networks show some advantages such that; it is not necessary to select a prediction method, great robustness to noises, etc. Therefore, it is understood that the neural networks present a superior or equal performance compared to the traditional statistical load forecasting methods. Furthermore it is a very promising technique and a good alternative to the applications in temporal series predictions area.

To illustrate the proposed method, an example is presented considering a historical data from a Brazilian Electric Company.

2. NEURAL NETWORK STRUCTURE

The *i*th output element (neuron) (Widrow and Lehr [21]) is a linear combination of the element inputs x_j that are connected to the element i by the bias w_{ij}:

$$\vartheta_i = \sum_j x_j \tag{1}$$

Each element can have a bias w_0 fed by an extra constant input $x_0 = +1$.

The linear output ϑ_i is finally converted in a non-linearity as a sigmoid, or a relay (Kartalopoulos [6], Widrow and Lehr [21]), etc.

3. NEURAL NETWORK TRAINING

The BP training is initialized by presenting a pattern $\boldsymbol{X}$ to the network that gives an output $\boldsymbol{Y}$. Following the calculated error in each output which is the difference between the desired value and the output. The next step is to determine the back propagated error by the network associated to the partial derivative of the quadratic error of each element related to the weights, and finally the weights of each element are adjusted. Then a new pattern is presented, and the process must be repeated until convergence is reached, where | error | ≤ a chosen tolerance. The initial weights are usually adopted as random numbers (Widrow and Lehr [21]). The BP algorithm consists in adapting the weights such that the network quadratic error is minimized. The sum of the instantaneous quadratic error of each neuron of the last layer (network output) is given by (Widrow and Lehr [17]):

$$\varepsilon^2 = \sum_{i=1}^{no} \varepsilon_i^2 \tag{2}$$

where

$\varepsilon_i = d_i - y_i$;
d_i = desired output of the ith element of the last layer ;
γ_i = output of the ith element of the last layer;
no = number of neurons of the last layer.

Considering the ith network neuron and using the descent gradient (Kartalopoulos [6], Widrow and Lehr [21]) method, the weight adjustments can be formulated by (Widrow and Lehr [21]):

$$\boldsymbol{V}_i (r+1) = \boldsymbol{V}_i (r) + \theta_i (r) \tag{3}$$

where

$\theta_i (r)$ = $\gamma [\nabla_i (r)]$;
γ = stability control parameter or training rate;
$\nabla_i (r)$ = quadratic error gradient related to neuron i weights;
$\boldsymbol{V}_i$ = vector containing neuron i weights = $[w_{0i}\ w_{1i}\ w_{2i} \ldots\ w_{ni}]^T$.

The adopted direction in equation (3) to minimize objective function of the quadratic error corresponds to the gradient opposite direction. The γ parameter determines the vector length $\theta_i(r)$. Providing that this work deals with load forecasting where the values are always positive, the nonlinear function to be used is the sigmoid function defined by Kartalopoulos [10], Widrow and Lehr [21] which is varying between 0 and +1 is given as:

$$y_i = 1 / \{(1 + \exp(-\lambda \vartheta_i)\} \tag{4}$$

where

λ = constant that determines the function y_i slope.

The BP algorithm can be found in the literature in several ways to become computationally faster. A very interesting formulation is the BP with momentum (Widrow and Lehr [21]).

Where the gradient is calculated as from equation (3), then by considering the sigmoid function defined in equation (4) and the momentum term, the following adaptation weight scheme is obtained (Minussi and Silveira [14], Widrow and Lehr [21]):

$$\boldsymbol{V}_{ij} (r+1) = \boldsymbol{V}_{ij} (r) + \Delta_{\boldsymbol{V}ij} (r) \tag{5}$$

where

$\Delta_{\boldsymbol{V}ij} (r)$ = $2\, y\, (1 - \eta)\, \beta_j\, x_i + \eta\ \Delta_{\boldsymbol{V}ij}$ (r– 1); (6)
$\boldsymbol{V}_{ij}$ = weight corresponding to the connection with the ith and the jth neuron;
η = momentum constant ($0 \le \eta \le 1$) (Widrow and Lehr [21]).

If the jth element is in the last layer, then:

$$\beta_j = \sigma_j\, \varepsilon_j \tag{7}$$

where

σ_j $\underline{\Delta}$ sigmoid function derivative, given by equation (4), related to:

$$\vartheta_i = \lambda y_j (1 - y_j) \tag{8}$$

If the *j*th element is in another layer, we have:

$$\beta_j = \sigma_j \sum_{k \in S(j)} w_{jk} \beta_k \tag{9}$$

where

$S(j)$ = set of the element subtitles that are in the next layer to the *j*th element layer and are interconnected to the *j*th element.

The γ parameter which is used as a stability control of the iterative process, depends of λ (Minussi and Silveira [14]).

The network weights are randomly initialized considering the interval {0,1}. By convenience, the parameter γ (training rate) can be redefined as:

$$\gamma = \gamma^* / (\lambda) \tag{10}$$

Substituting equation (10) into equation (6), the amplitude dependency of σ related to λ. is cancelled. The σ amplitude will be maintained constant to every λ. This alternative is important considering that λ will only actuate in the left and right tails of σ. Thus, equation (6) can be written as:

$$\Delta_{Vij}(r) = \{ 2 \gamma^* (1 - \eta) \beta_j / \lambda \} x_i + \eta (V_{ij} (r - 1) \tag{11}$$

The BP training can be done by two ways.

- *Procedure 1*
  Adjusting the network weights (considering all layers), converging each pattern until the input pattern set is completed. The process will be repeated until total convergence occurs, i.e. the quadratic error will be less than a predefined tolerance taking into account the given pattern.
- *Procedure 2*
  It is the same as the first one, but doing only one iteration per pattern (weight adjusting).

4. A FUZZY LOGIC APPROACH

This work proposes an approach to adjust the training rate γ^* during the convergence process which results a reduction of the training time execution. The γ^* adjustment is done by a proceeding based on a fuzzy controller. The basic idea of the methodology consists of determining the system state, defined as the global error εg and the global error variation $\Delta\varepsilon g$, objecting a control structure that leads the error to be zero in a reduced iteration number, when compared to the conventional procedures. In this work, the control is formulated using the fuzzy logic concepts. The global error εg and its variation $\Delta\varepsilon g$ are the system state components, and $\Delta\gamma^*$ is the control action that must be done in the system. Initially, the global error is defined as:

$$\varepsilon g = \sum_{j=1}^{np} \sum_{j=1}^{np} \varepsilon_i^2 \tag{12}$$

where

εg = global error of the neural network;
np = number of the network pattern vectors.

The global error corresponds to the calculation of all output errors, output neurons, considering all network pattern vectors. The training must be executed using procedure 2, one iteration per pattern. The global error is calculated in each iteration and the parameter γ^*, is adjusted by increasing $\Delta\gamma^*$ determined by fuzzy logic. The system state and the control action are defined as:

$$E^q = [\varepsilon g \ \Delta\varepsilon g^q \ ] \ T \tag{13}$$
$$u^q = \Delta\gamma^{*q} \tag{14}$$

where q = the index that gives the current iteration.

For a very big input pattern $\boldsymbol{X}$, εg and $\Delta\varepsilon g$ can saturate. Then, the adaptive control is done using an exponential decreasing function applied to the fuzzy controller response. In such a way, the adaptive controller is given by:

$$\Delta\gamma^{*q} = \exp(-\alpha q) \ \Delta\psi^q \tag{15}$$

where

α = an arbitrary positive number;
$\Delta\psi^q$ = the increase proceeding from the fuzzy controller at the instant q.

This parameter will be used to adjust the network weight set referred to the subsequent iteration.

The process is repeated until the training is completed. This is a very simple procedure whose control system requires an additional effort, providing that the controller has two input variables and only one output. This is an improvement to that presented in Reference. [2], i.e. by using the same variables εg and $\Delta\varepsilon g$ to do the control. However, this work introduces the following contributions; 1) improvements of BP algorithm are gained; and 2) the proposed fuzzy controller is a new one, where a set of rules and the use of an exponential decreasing function applied to the controller response.

Definition. Consider a set of objects *B*. Then, a fuzzy set *A* in *B* is defined as being a set of coordinated pairs (Terano *et al*. [18]):

$$A = \{(b, \mu_A(b)) \mid b \in B\} \tag{16}$$

where

$\mu_A(b)$ = membership function of *b* in *A*.

The membership function $\mu_A(b)$ shows the degree of how *b* is in *A*.

The fuzzy control is basically formed by three parts: *fuzzification* that converts real variables in linguistic variables; *inference* that consists of the manipulation of rules using *if-then* statements, and fuzzy operations (AND, OR, NOT, etc.) and *defuzzification* that converts the results obtained (linguistic variables) in real variables, which

form the control action. The fuzzy membership functions can have three different forms, such as: triangular, trapezoidal and gaussian, according to the preference/experience of the designer. The most common defuzzification method is the centroid (Terano *et al*. [18]) that finds the inertial center of the fuzzy set solution. We have the following equation that consider a discrete fuzzy set (Terano *et al*. [18]):

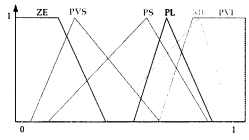
$$u = \left(\sum_{i=1}^{nr} \mu_i \, a_i\right) / \left(\sum_{i=1}^{nr} \mu_i\right) \qquad (17)$$

where

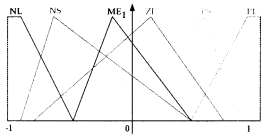
μ_i = value of the membership function;
a_i = value of the set that contains a membership value;
nr = number of fuzzy rules.

The u value calculated by equation (17) corresponds to the inertial center projection of the figure defined by the rule set over the control variable.

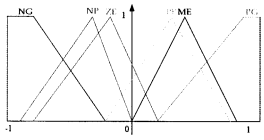
Each state variable must be represented between 3 and 7 fuzzy sets. The control variable must also be represented by the same number of fuzzy sets. The εg variable must be normalized considering as a schedule factor the first global error generated by the network, i.e. $q = 0$. With this representation, the variation interval will be between 0 and +1. If the adaptation heuristic is accordingly coincident the process convergence will be an exponential decreasing. The $\Delta\varepsilon g$ variable will vary between –1 and +1. If the convergence process is exponential decreasing, the $\Delta\varepsilon g$ values will always be negative. In this case, although the $\Delta\varepsilon g$ schedule is between –1 and +1, it must be employed in the rule set, an accurate adjustment between – 1 and 0. In the other interval (0, +1], the adjustment can be more relaxed. The membership functions of the fuzzy sets for several variables used in the neural network BP fuzzy controlled training are shown in Figure 1.



(a)Membership function for the global error εg.



(b)Membership function for the global error variation $\Delta\varepsilon g$.



(c)Membership function for the control action $\Delta\psi$.

Figure 1 Membership functions for the variables εg, Δεg and Δα. Where:
NL = negative large
NS = negative small
ZE= near to zero
PVS = positive very small
ME = medium
PL = positive large
PVL= positive very large
lu = variation limit of Aw

In fuzzy controller the rules are modified as a decision table form. Each input represents the fuzzy value of the variable $\Delta\psi$ given the global error values εg and the global error variation $\Delta\varepsilon g$. The parameter γ^* should be arbitrated in function of λ (sigmoid function slope). The variations γ^* also follow the same procedure. Then, the *lu* parameter must be adjusted to attend each problem specifications and in function of λ.

Table 1 shows the fuzzy rules set in a total of 30 rules. The number of rules can be increased to improve the network performance during the training. This Table is done based on the experience of the user according to references (John [8], Kaehler [9] and Terano *et al.* [18]).

5. LOAD FORECASTING FORMULATION

Short term load forecasting (daily forecasting) is done according to the following: implementation of a recurrence in the output of a determined instant will be used as an input on the subsequent instant. There are considered the hourly historic data in a predefined interval, e.g. monthly. The input network to an hour h is defined as the load val-

Table 1 Fuzzy controller rules

1	if	εg	is	*ZE*	and	$\Delta\varepsilon g$	is *NL*,	then	$\Delta\psi$	is *ZE*
2	if	εg	is	*ZE*	and	$\Delta\varepsilon g$	is *NS*,	then	$\Delta\psi$	is *NL*
3	if	εg	is	*ZE*	and	$\Delta\varepsilon g$	is *ZE*,	then	$\Delta\psi$	is *PS*
4	if	εg	is	*ZE*	and	$\Delta\varepsilon g$	is *PS*,	then	$\Delta\psi$	is *PL*
5	if	εg	is	*ZE*	and	$\Delta\varepsilon g$	is *ME*,	then	$\Delta\psi$	is *ZE*
6	if	εg	is	*PVS*	and	$\Delta\varepsilon g$	is *NL*,	then	$\Delta\psi$	is *ZE*
7	if	εg	is	*PVS*	and	$\Delta\varepsilon g$	is *NS*,	then	$\Delta\psi$	is *ZE*
8	if	εg	is	*PVS*	and	$\Delta\varepsilon g$	is *ZE*,	then	$\Delta\psi$	is *NL*
9	if	εg	is	PVS	and	$\Delta\varepsilon g$	is *PS*,	then	$\Delta\psi$	is *ME*
10	if	εg	is	*PVS*	and	$\Delta\varepsilon g$	is *PL*,	then	$\Delta\psi$	is *ZE*
11	if	εg	is	*PS*	and	$\Delta\varepsilon g$	is *NL*,	then	$\Delta\psi$	is *NS*
12	if	εg	is	*PS*	and	$\Delta\varepsilon g$	is *NS*,	then	$\Delta\psi$	is *ME*
13	if	εg	is	*PS*	and	$\Delta\varepsilon g$	is *ZE*,	then	$\Delta\psi$	is NL
14	if	εg	is	*PS*	and	$\Delta\varepsilon g$	is *ME*,	then	$\Delta\psi$	is *ZE*
15	if	εg	is	*PS*	and	$\Delta\varepsilon g$	is *PL*,	then	$\Delta\psi$	is *NL*
16	if	εg	is	*ME*	and	$\Delta\varepsilon g$	is *NL*,	then	$\Delta\psi$	is *PL*
17	if	εg	is	*ME*	and	$\Delta\varepsilon g$	is *NS*,	then	$\Delta\psi$	is *NL*
18	if	εg	is	*ME*	and	$\Delta\varepsilon g$	is *PS*,	then	$\Delta\psi$	is *PL*
19	if	εg	is	*ME*	and	$\Delta\varepsilon g$	is *ME*,	then	$\Delta\psi$	is *PS*
20	if	εg	is	*ME*	and	$\Delta\varepsilon g$	is *PL*,	then	$\Delta\psi$	is *ME*
21	if	εg	is	*PL*	and	$\Delta\varepsilon g$	is *NS*,	then	$\Delta\psi$	is *ME*
22	if	εg	is	*PL*	and	$\Delta\varepsilon g$	is *ZE*,	then	$\Delta\psi$	is *PS*
23	if	εg	is	*PL*	and	$\Delta\varepsilon g$	is *PS*,	then	$\Delta\psi$	is *NS*
24	if	εg	is	*PL*	and	$\Delta\varepsilon g$	is *ME*,	then	$\Delta\psi$	is *ZE*
25	if	εg	is	*PL*	and	$\Delta\varepsilon g$	is *PL*,	then	$\Delta\psi$	is *ME*
26	if	εg	is	*PVL*	and	$\Delta\varepsilon g$	is *NL*,	then	$\Delta\psi$	is *PL*
27	if	εg	is	*PVL*	and	$\Delta\varepsilon g$	is *ZE*,	then	$\Delta\psi$	is *NS*
28	if	εg	is	*PVL*	and	$\Delta\varepsilon g$	is *PS*,	then	$\Delta\psi$	is *ZE*
29	if	εg	is	*PVL*	and	$\Delta\varepsilon g$	is *ME*,	then	$\Delta\psi$	is *NL*
30	if	εg	is	*PVL*	and	$\Delta\varepsilon g$	is *PL*,	then	$\Delta\psi$	is *ZE*

ues, extracted from the historical data in four instants (current value, one, two and three hours before), temperature, etc., and data referred to time (month, day of the week, holiday and hour, etc.). The output network corresponds to the load value referred to the hour (h + 1). The output/input set is defined considering this strategy until all time series interval time is completed. This scheme can be modified to improve the results by introducing other variables (nebulous day, etc.). Then the input and output vectors are respectively defined as following:

$$\boldsymbol{X}(h) = [\, \boldsymbol{t}^T \, L(h-3)\, L(h-2)\, L(h-1)\, L(h)]^T \,,\ \boldsymbol{X} \in R^m \quad (18)$$

$$\boldsymbol{Y}(h) = [\, \mathrm{L}(h+1)\,] \,,\ \boldsymbol{Y} \in R^l \quad (19)$$

where:

m = dimension of vector $\boldsymbol{X}$;

$L(h-\mathrm{p})$ = the load value at the p hours before the current hour h;

$L(h+1)$ = electric load value corresponding to the subsequent hour to current hour h;

t = time vector referred to the historic data (month, day of the week, holiday, hour, etc) represented in a similar way as the binary code (– 1,+1).

The adoption of a binary representation is preferable in relation to (0, +1) representation, considering that the network input component "0" does not modify the weights. Using this approach the (–1, +1) representation gives a faster convergence and consequently being more efficient.

The electric loads $L(h-3)$, ... , $L(h-1)$ represent the feedback link, with a delay in the output. Then, this network is a recurrent one, and is considered a sigmoid function defined by equation (4). In the BP algorithm, the adaptation of the weights that connect to each neuron is carried out using, basically, the error propagated in the inverse direction, multiplied by $\sigma(\vartheta)$ (partial derivative of y with respect to ϑ) and the input in the referred neuron. In this way, the weights have effective adjustment only for values of ϑ situated in the *central body* of $\sigma(\vartheta)$ function. As the weights become significant, high or low a deceleration of adjustments occurs, having a tendency to a complete paralysis, exactly where the ends of both *right* and *left* tails of $\sigma(\vartheta)$ are met together. In view of this, in the specific case of the problem of electric load forecasting determination, it is proposed the use of sigmoid function relatively small λ (inferior to 1), providing longer tails (Minussi and Silveira [14]). This will permit a less restrictive choice of the network weights, if compared to the adopted in the bibliography. In such a way this reduces the possibility of occurrence of paralysis and increases the speed of the BP algorithm convergence (Minussi and Silveira [14]).

The training data used in this work are (for each vector): the time data, month day, week day (if it is a holiday or not), day hour, the current hour loads, and load values considering the previous three hours. The future load (one hour in advance) is the network output. There are not considered the temperature data because they are not available. Considering a binary representation, the vector **t** will have a dimension of 8, that joining the load data of 12 components (m = 12). The historical data are electric loads and are obtained from a Brazilian Electricity Company. These data contain hourly loads of the year 1998, that related to non typical days (holidays), special days (Saturdays and Sundays) and days from a typical week.

In our experience we consider two different artificial neural networks designated by neural network 1 and neural network 2. Neural network 1 is composed by the historic data between June 17th, 1998 and July 28th, 1998. Therefore, there are 1008 input/output vectors. Table 2 shows the principal parameters referred to the used neural network and the training while Figure 2 shows the results of load forecasting (by conventional BP and fuzzy controlled BP) referred to the defined day July 29th, 1998. Neural network 2 formed by the historic data between May 2th, 1998 and July 25th, 1998 is composed by 2016 input/output vectors. In Table 2 it is verified that the principal parameters referred to the used neural network and the training while Figure 3 shows the results of load forecasting (by conventional BP and fuzzy controlled BP) referred to the defined day July 26th, 1998.

For a precision analysis, we define the mean absolute percentage error (*MAPE*) (Srinivasan et al. [16]) and the maximum error of the daily forecasting, comparing the real load values with the estimated values by neural network in the following way:

$$MAPE = \frac{1}{N} \{ \sum_{i=1}^{N} (|\, L(h) - \underline{L}(h)|\,) \,/\, L(h) \} \times 100 \quad (20)$$

$$Maximum\ error\ (\%) = max\{|\, L(h) - \underline{L}(h)\,| \,/\, L(h)\} \times 10 \quad (21)$$

Table 2 Neural network specification

Item	Neural Network 1	Neural Network 2
Pattern vectors number	1008	2016
Layers number	3	3
Neurons number per layer	12-30-1	12-30-1
Tolerance	0.025	0.030
Training rate γ*	5.5	6.0
Moment term η	0.9	0.9
Inclination of sigmoid function λ	0.3	0.3
Parameter α	0.81	0.4

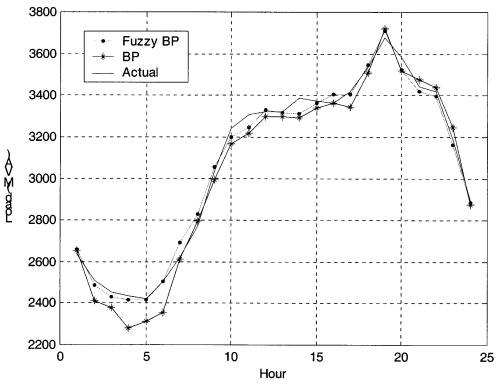


Figure 2 Load forecasting results to neural network 1

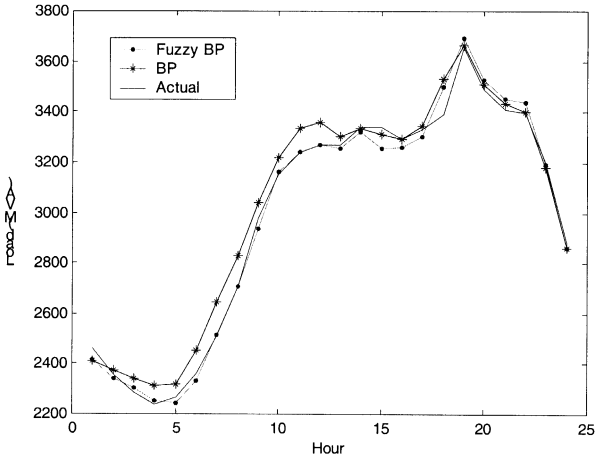


Figure 3 Load forecasting results to neural network 2

where

$L(h)$ = real load value referred to hour h;
$\underline{L}(h)$ = estimated load value referred to hour h;
N = total number of hours.

Some other forecasting are done by considering several load sets. In all cases, it was observed that the fuzzy controlled backpropagation neural network shows a better performance, either in precision or in speed, when compared to the results obtained from a conventional backpropagation, according to Tables 3 and 4. It is also observed that, for obtaining a similar forecasting, using the traditional backpropagation, it is necessary to reduce the tolerance of the iterative process.

6. CONCLUSION

A methodology for electric load forecasting is developed using a neural network The training is done based on a fuzzy controller BP algorithm. The presented results consider the historical data of a Brazilian Electricity Company. The short term load forecasting is done considering 24 hours in advance. It can be verified using these results, that the proposed formulation presents a superior performance, regarding the precision and speed, when compared to conventional BP algorithm. The observed mean absolute percentage error (*MAPE*) and the maximum error of the daily forecasting was 0.91% and 2.71%, respectively. Using the traditional backpropagation, the observed errors were respectively, 1.97% and 6.36% for neural network 1. The MAPE and the Maximum Error obtained on neural network 2 were 0.94% and 3.20%, respectively, while on conventional backpropagation these errors were approximately around 1.85% and 5.52%, respectively.

Table 3 Comparitive results to application 1

Item	Conventional BP	BP with fuzzy controller
Number of cycles	2452	1932
Processing time (s)	221.95	179.12
MAPE (%)	1.97	0.91
Maximum error (%)	6.36	2.71

Table 4 Comparitive results to application 2

Item	Conventional BP	BP with fuzzy controller
Number of cycles	1351	1455
Processing time (s)	98.49	103.49
MAPE (%)	1.85	0.94
Maximum error (%)	5.52	3.20

ACKNOWLEDGMENTS

The authors would like to acknowledge the financial support of FAPESP (Fundação de Amparo à Pesquisa do Estado de São Paulo) – Brazil (Proc. No. 98/03332-2 and No. 00/15120-1).

REFERENCES

1 Almeida C, Fishwich P.A. and Tang Z. Time Series Forecasting Using Neural Network vs. Box-Jenkins Methodology, *Simulation* (1991) pp. 303–310
2 Arabshahi P, Choi J.J, Marks II R. J. and Caudell T. P. Fuzzy Parameter Adaptation in Optimization, *IEEE Computational Science & Engineering* (1996) pp. 57–65
3 Bunn, D. W. Forecasting Loads and Prices in Competitive Power Markets, *Proceedings of IEEE*, Vol 88 No 2 (2000) pp. 163–169
4 Charytoniuk, W. and Chen, M-S. Very Short-Term Load Forecasting Using Artificial Neural Networks, *IEEE Transactions on Power Systems*, Vol 15 No 1 (2000) pp. 263–268
5 O'Donovan T. M. *Short Term Forecasting: An Introduction to the Box-Jenkins Approach*, John Wiley & Sons, New York, USA (1983)
6 Doulai, P. and Cahill, W. Short-term Price Forecasting in Electric Energy Market, *The Fifth International Power Engineering Conference*, (may 2001) pp. 749–754
7 Hippert, H. S., Pedreira, C. E. and Souza, R. C. Neural Networks for short-term load forecasting: A review and evaluation,*IEEE Transactions on Power Systems*, Vol 16 No 1 (2001) pp. 44–55
8 John, R. I. Fuzzy Inferencing System – Problems and Some Solutions, *CMS Working Paper* no. 62, School of Computing Sciences of Montfort University (December 1995)
9 Kaehler, S. D. Fuzzy Logic Tutorial – An Introduction, ENCODER – *The Newsletter of the Seattle Robotics Society* (March 1998)
10 Kartalopoulos S.V. Understanding Neural Networks and Fuzzy Logic, *IEEE Press,* New York, USA (1996)
11 Kodogiannis V.S. and Anagnostakis E. M. A Study of Advanced Learning Algorithms For Short-term Load Forecasting, *Engineering Applications of Artificial Intelligence,* Vol 12 (1999) pp. 159–173
12 Kodogiannis V.S. and Anagnostakis E. M. Soft Computing Based Techiniques For Short-term Load Forecasting, *Fuzzy Sets and Systems*, Vol 128 (2002) pp. 413–426
13 Liang R-H. and Cheng C-C. Short-term Load Forecasting by a Neuro-fuzzy Based Approach, *Electrical Power and Energy Systems*, Vol 24 (2002) pp. 103–111
14 Minussi C.R. and Silveira M.C.G. Electric Power System Transient Stability By Neural Networks, *38rd Midwest Symposium On Circuits and System* (1995) pp. 1305–1308
15 Osowski S. and Siwek K. Regularisation of Neural Networks

For Improved Load Forecasting in The Power System, *IEE Proceedings Generation, Transmission and Distribution*, Vol 149 No 3 (May 2002) pp. 340–344

16 Srinivasan D., Tan S. S., Chang C. S. and Chan E.K. Practical Implementation of a Hybrid Fuzzy Neural Network For One-Day-Ahead Load Forecasting, *IEE Proceedings Generation, Transmission and Distribution* Vol 145 No 6 (1998) pp. 687–692

17 Tamimi, M. and Egbert, R. Short term load forecasting via fuzzy neural collaboration, *Electric Power Systems Research* Vol 56 (2000) pp. 243–248

18 Terano T., Asai K. and Sugeno M. *Fuzzy Systems Theory and Its Applications*, Academic Press, New York, USA (1991)

19 Wang, X., Hatziargyriou, N. and Tsoukalas, L. H. A New Methodology for Nodal Load Forecasting in Deregulated Power Systems, *IEEE Power Engineering Review*, Vol 22 No 5 (2002) pp. 48–51

20 Werbos P.J. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences, *Master Thesis*, Harvard University, Cambridge, MA, USA, (1974)

21 Widrow B. and Lehr M. A. 30 Years of Adaptive Neural Networks: Perceptron, Madaline, and Backpropagation, *Proceed-*

ings of the IEEE, Vol 78 (1990) pp. 1415–1442